A C M - X C P C

算法模板合集

竞赛代码模板 · header-only

数据结构 · 图论 · 数学 · 字符串 · 对拍

ihxnan

目 录

模板骨架 5

精简版骨架 Template/contest.hpp 8

Python 版骨架 Template/ihxnan.py 10

数据结构 11

DSU 并查集 DSU.hpp 12

树状数组 Fenwick.hpp 13

线段树（区间加 / 区间求和）SegmentTree.hpp 14

ST 表（稀疏表）ST.hpp 16

整数位封装 Bit.hpp 17

FHQ Treap（数组版，按值 / 按大小分裂）Treap.hpp 18

FHQ Treap（指针版，免预估节点数）TreapPtr.hpp 21

莫队 Mo.hpp（片段式） 24

图论 29

最大流（Dinic）MaxFlow.hpp 30

费用流（SPFA 朴素版，默认）MinCostFlow.hpp 33

费用流（Dijkstra + 势优化，被卡时间再换）MinCostFlowDijkstra.hpp 36

强连通分量（Tarjan）SCC.hpp 38

边双连通分量（EBCC）EBCC.hpp 40

圆方树（点双 → 块割树）BlockCutTree.hpp 42

2-SAT TwoSat.hpp 44

二分图最大匹配（匈牙利）Hungarian.hpp 45

二分图最大权完美匹配（KM）MaxAssignment.hpp 46

重链剖分（HLD，1-indexed）HLD.hpp 49

数学 51

组合数（阶乘 + 逆元，按需扩表）Comb.hpp 52

欧拉线性筛（素数表）EulerPrime.hpp 54

扩展欧几里得（解不定方程）Exgcd.hpp 55

Miller-Rabin 素性测试 MillerRabin.hpp 57

Pollard-Rho 质因数分解（含 Miller-Rabin）PollarRho.hpp 58

线性基 LinearBasis.hpp 60

计算几何（点 / 线 / 多边形 / 半平面交）Point.hpp 62

字符串 69

KMP / Z 函数（1-indexed）KMP.hpp 70

字符串哈希（自然溢出，0-indexed）StrHash.hpp 71

Trie（62 字符集，支持计数 / 删除）Trie.hpp 72

AC 自动机（小写字母）AhoCorasick.hpp 74

子序列自动机 SequenceAM.hpp 76

后缀数组（倍增 + 计数排序，0-indexed）SuffixArray.hpp 77

对拍 79

Windows 对拍脚本 dp.bat 80

模板骨架 Template/ihxnan.hpp

用法：开题时把本文件整体粘贴到提交文件最上方，在 solve() 内写题解。

- 默认单组用例（t = 1），多组用例时在 solve() 开头读入 t 即可（main 里 while (t--)）。
- cin / cout 已重定向到快读快写 fin / fout；**仅支持整数、char、string**（double 等类型编译期报错，与提交模式一致，避免"本地能编、评测报错"）。
- 本地调试编译需 -std=c++20 -DIHXNAN（dbg.hpp 依赖 C++20 source_location），提供 gdb() 变量打印、gdbif() 条件打印、gdb_assert() 断言、TIME() 计时；提交时不加该宏，仅 gdb() / TIME() 退化为空操作 —— **gdbif() / gdb_assert() 在提交版会编译错误**，需删除或包进 #ifdef IHXNAN。
- 快写基于缓冲区，程序结束由析构统一 flush，勿在中间混用 printf。
- rd0(x) 一行整体读入容器（vector / array / deque 等，不含 map 嵌套），rd1(x) 从 x[1] 读到末尾（适配 1-indexed 容器）；分块宏 bl / bi / br 需先定义 blen（br 还需 n），fid / gx / gy 需先定义 m。
- iINF / mod / lINF / MOD 均为 const ll，**不可再赋值**（双模数等场景请自定义变量）。
- fin 读到 EOF 后 ok 置 false，可用 if (fin) 判断输入是否结束（适合未给组数的多组输入）。

```
#include <bits/stdc++.h>

// ============ 常用宏 ============
#define endl '\n'                                // 输出换行（配合下方 IO 结构体，末尾无多余空白）
#define ls(x) ((x) << 1)                         // 线段树/树状结构：节点 x 的左儿子下标（x*2）
#define rs(x) ((x) << 1 | 1)                     // 线段树/树状结构：节点 x 的右儿子下标（x*2+1）
#define bl(x) ((x) * blen + 1)                   // 分块：块 x 左边界下标（1 起始，含；需已定义 blen）
#define bi(x) (((x) - 1) / blen)                 // 分块：下标 x 所属块编号（0 起始；需已定义 blen）
#define br(x) (min(((x) + 1) * blen, n))         // 分块：块 x 右边界下标（受全局 n 限制，含；需已定义 bl
en、n）
#define fid(x, y) ((m) * (x - 1) + (y))          // 二维坐标 (x,y) → 一维编号（1-indexed；需已定义 m）
#define gx(id) (((id) - 1) / m + 1)              // fid 逆变换：编号 → 行号（需已定义 m）
#define gy(id) (((id) - 1) % m + 1)              // fid 逆变换：编号 → 列号（需已定义 m）
#define rd0(x) for (auto &p : x) cin >> p        // 容器整体读入（vector/array/deque 等，不含 map 嵌套）
#define rd1(x) for (int i = 1; i < x.size(); ++i) cin >> x[i] // 从 x[1] 读到末尾（1-indexed 容器，跳过
x[0]）

// ============ 主框架 ============
// t = 用例数（默认 1，多组题在 solve() 开头 cin >> t）；solve() 由选手实现
int t = 1; void solve();
int main() { while (t--) solve(); return 0; } // 循环 t 次调用 solve()

// ============ 类型别名 ============
using namespace std;
using ll = long long;                  // 64 位有符号整数（默认整型）
using ul = unsigned long long;         // 64 位无符号整数（自然溢出哈希等）
using lll = __int128;                  // 128 位整数（GCC 扩展；打印需 -DIHXNAN 的 dbg.hpp）
using ld = long double;                // 高精度浮点（快读快写不支持，避免 cout）
using vb = vector<bool>;               // bool 向量（位压缩，取址行为特殊）
using vi = vector<int>;                // int 向量
using vvi = vector<vi>;                // 二维 int 向量
using vl = vector<ll>;                 // ll 向量
using vvl = vector<vl>;                // 二维 ll 向量
using pii = pair<int, int>;            // int 二元组
using pll = pair<ll, ll>;              // ll 二元组
using ti = tuple<int, int, int>;       // int 三元组（如查询 (l, r, id)）
using tl = tuple<ll, ll, ll>;          // ll 三元组

// ============ 常量（const ll，不可再赋值）============
const ll iINF = 0x3f3f3f3f,            // int 上无穷大（≈1.06e9，相加不易溢出）
         mod = 998244353,              // 常用 NTT 模数
         lINF = 0x3f3f3f3f3f3f3f3f,    // ll 上无穷大（≈4.56e18，相加不易溢出）
```

```
    MOD = 1000000007;            // 常用质数模数

#ifdef IHXNAN
    // 本地调试模式：dbg.hpp 提供同逻辑 IO 与调试宏（需先 symlink；编译加 -std=c++20）
    #include <dbg>
#else
    // 提交模式：cin/cout 重定向到快读快写；调试宏仅 gdb/TIME 有空操作回退
    #define cin fin
    #define cout fout
    #define gdb(...)   ((void)0)
    #define TIME(name) ((void)0)

    struct IO {
        static const int S = 1 << 21; // 输入/输出缓冲区大小
        uint8_t buf[S], *p1, *p2;
        int st[105], Top;             // st 为输出数位栈
        bool ok;                      // 读到 EOF 后置 false，可用 if (fin) 判断
        IO() : p1(nullptr), p2(nullptr), Top(0), ok(true) { st[0] = 0; }
        ~IO() { clear(); }
        inline void clear() { fwrite(buf, 1, Top, stdout); Top = 0; }

        // 读一个字节（缓冲区空则批量 fread）
        inline int inc() {
            if (p1 == p2) {
                p2 = (p1 = buf) + fread(buf, 1, S, stdin);
                if (p1 == p2) return EOF;
            }
            return *p1++;
        }
        // 写一个字节（缓冲区满则落盘）
        inline void outc(int c) {
            if (Top == S) clear();
            buf[Top++] = (uint8_t)c;
        }
        inline explicit operator bool() const { return ok; }

        // 输出整数（仅算术类型）
        template <typename T>
        inline IO &operator<<(T x) {
            int f = 0;
            if constexpr (std::is_signed_v<T>) { if (x < 0) outc('-'), f = 1; }
            do { st[++st[0]] = f ? -(x % 10) : x % 10, x /= 10; } while (x);
            while (st[0]) outc(48 ^ st[st[0]--]);
            return *this;
        }
        // 输出字符 / 字符串
        inline IO &operator<<(char c) { outc(c); return *this; }
        inline IO &operator<<(const char *x) { while (*x) outc(*x++); return *this; }
        inline IO &operator<<(char *x) { while (*x) outc(*x++); return *this; }
        inline IO &operator<<(const string &x) { for (auto c : x) outc(c); return *this; }

        // 读入整数（跳过非数字，支持负号；EOF 置 ok=false）
        template <typename T>
        inline IO &operator>>(T &x) {
            x = 0;
            int f = 0, ch = inc();
            while (ch != EOF && !isdigit(ch)) { if (ch == '-') f ^= 1; ch = inc(); }
            if (ch == EOF) { ok = false; return *this; }
            while (ch != EOF && isdigit(ch)) x = (x << 3) + (x << 1) + (ch ^ 48), ch = inc();
            f ? x = -x : 0;
            return *this;
        }
        // 读入单个字符（跳过空白）
        inline IO &operator>>(char &x) {
```

```
            int c = inc();
            while (c != EOF && isspace(c)) c = inc();
            if (c == EOF) { ok = false; return *this; }
            x = (char)c;
            return *this;
        }
        // 读入空白分隔的字符串
        inline IO &operator>>(string &x) {
            x.clear();
            int c = inc();
            while (c != EOF && isspace(c)) c = inc();
            if (c == EOF) { ok = false; return *this; }
            while (c != EOF && !isspace(c)) x += char(c), c = inc();
            return *this;
        }
    } fin, fout;
#endif
```

精简版骨架 Template/contest.hpp

> **用法**：正式赛打印后手抄用，比 ihxnan.hpp 短很多：快读改用 ios::sync_with_stdio(false) + cin.tie(nullptr)，调试为内联的简易 gdb，不再 include Debug/dbg.hpp。
>
> - **默认多组用例**：main 内 int t = 1; cin >> t; 无条件读入组数；单组题请注释掉 cin >> t;。
> - 调试编译 g++ -std=c++17 -O2 -DIHXNAN；提交 g++ -std=c++17 -O2（不加宏时 gdb(...) 展开为 void()）。
> - gdb() 只支持有 operator<< 的类型（整数、char、string 等）；**vector/map/set/pair/ __int128 等无法直接打印**，变量名含逗号（如 pair<int,int>、f(1,2)）会被切错。
> - 保留宏 ls / rs / bl / bi / br / fid / gx / gy；未提供 rd0 / rd1。

```
#include <bits/stdc++.h>

#define endl '\n'                               // 输出换行，配合下方 IO 结构体使用（末尾无多余空白）
#define ls(x) ((x) << 1)                        // 线段树/树状结构：节点 x 的左儿子下标（x*2）
#define rs(x) ((x) << 1 | 1)                    // 线段树/树状结构：节点 x 的右儿子下标（x*2+1）
#define bl(x) ((x) * blen + 1)                  // 分块：块 x 的左边界下标（1 起始，含）
#define bi(x) (((x) - 1) / blen)                // 分块：下标 x 所属的块编号（0 起始，blen 为块长）
#define br(x) (min(((x) + 1) * blen, n))        // 分块：块 x 的右边界下标（受全局 n 限制，含）
#define fid(x,y) ((m)*(x-1)+(y))                // 二维坐标 (x,y) → 一维编号（1-indexed，m 为列数）
#define gx(id) (((id) - 1) / m + 1)             // fid 逆变换：编号 id → 行号 x（1-indexed）
#define gy(id) (((id) - 1) % m + 1)             // fid 逆变换：编号 id → 列号 y（1-indexed）

// ============ 类型别名（与 ihxnan.hpp 完全一致，挤成一行省篇幅）============
using namespace std;using ll=long long;using ul=unsigned long long;using lll=__int128;using ld=long doub
le;using vb=vector<bool>;using vi=vector<int>;using vvi=vector<vi>;using vl=vector<ll>;using vvl=vector<
vl>;using pii=pair<int, int>;using pll=pair<ll,ll>;using ti=tuple<int,int,int>;using tl=tuple<ll,ll,ll>;

// ============ 常量 ============
const ll iINF=0x3f3f3f3f,mod=998244353,lINF=0x3f3f3f3f3f3f3f3f,MOD=1000000007;

#ifdef IHXNAN
    // gdb(a, b) 依次打印 "a = ... , b = ..."（变量名文本与值交替输出到 stderr）
    template <typename T> void _debug(char *f, T t) // 递归终点：只剩一个变量
    {
        cerr << f << " = " << t << endl;
    }
    template <typename T, typename... Ar> void _debug(char *f, T x, Ar... y) // 变参递归
    {
        while (*f != ',')          // 打印变量名部分（直到逗号）
            cerr << *f++;
        cerr << " = " << x << ",";  // 打印 " = 值,"
        _debug(f + 1, y...);        // 跳过逗号，递归处理剩余变量
    }
    #define gdb(...) _debug((char *)#__VA_ARGS__, __VA_ARGS__) // #__VA_ARGS__ 取变量名文本
#else
    #define gdb(...) void() // 提交模式：展开为空语句
#endif

void solve() // 由选手实现
{

}

int main()
{
    ios::sync_with_stdio(false); // 关闭与 C 标准库的同步，加速 cin/cout
    cin.tie(nullptr);            // 解除 cin 与 cout 绑定，避免每次读入前 flush
    cout.tie(nullptr);
    int t = 1;
    cin >> t;                    // 默认多组：无条件读入组数（单组题请注释掉本行）
```

```
    while (t--)
        solve();
    return 0;
}
```

Python 版骨架 `Template/ihxnan.py`

> 快速 IO 辅助函数：`I()` 读一行去空白、`II()` 读整数、`MII()` 读一行整数 map、`LII()` 读一行整数列表、`GMI()` / `LGMI()` 为 0-indexed 版（整体减一）。

```
import math
import sys
from collections import Counter, defaultdict, deque  # 常用容器
from heapq import heapify, heappop, heappush         # 堆（优先队列）

input = sys.stdin.readline       # 用 C 层 readline 替代 input，读整行更快
sys.setrecursionlimit(1000000)   # 抬高递归上限，避免深搜/递归爆栈


def I():                         # 读一行，去掉首尾空白（字符串）
    return input().strip()


def II():                        # 读一个整数（整行）
    return int(input())


def MII():                       # 读一行整数，返回 map（可 `a, b = MII()` 解包）
    return map(int, input().split())


def LII():                       # 读一行整数，返回 list
    return list(map(int, input().split()))


def GMI():                       # 同 MII，但整体减一（0-indexed 建图用）
    return map(lambda x: int(x) - 1, input().split())


def LGMI():                      # 同 LII，但整体减一
    return list(map(lambda x: int(x) - 1, input().split()))
```

数据结构 DataStructure/

> 约定：默认 **1-indexed**（构造传点数/元素数 `n`，内部开 `n+1`）。 依赖 `Template/ihxnan.hpp` 的类型别名（`ll`、`vi`、`pii` 等）；`SegmentTree` 还依赖宏 `ls` / `rs`。

DSU 并查集 DSU.hpp

- **用法**：DSU dsu(n)（点号 1..n）；merge(x, y) 合并，返回是否真的合并（可用于 Kruskal 计数）；same(x, y) 判连通；size(x) 返回 x 所在连通块大小。
- **实现**：路径减半 + 按大小合并，均摊近 O(α)。
- **注意**：下标 0 也存在（自根），但按约定从 1 开始使用。

```
struct DSU
{
    vector<int> fa, sz; // fa[i]: i 的父节点（根满足 fa[i]==i）；sz[i]: i 为根时连通块大小

    // 构造 n 个点（编号 1..n，内部开 n+1），每个点先各自成根
    DSU(int n) : fa(n + 1), sz(n + 1, 1)
    {
        iota(fa.begin(), fa.end(), 0); // fa[i] = i，初始自为根
    }

    // 查根（路径减半：边走边把 fa 指向祖父，均摊近 O(α)）
    int get(int x)
    {
        while (x != fa[x])
            x = fa[x] = fa[fa[x]]; // 压缩：把当前点直接连到祖父
        return x;
    }

    // 合并 x、y 所在集合，返回是否真的合并（同集合返回 false）
    bool merge(int x, int y)
    {
        if ((x = get(x)) == (y = get(y))) // 先各取根
            return false;
        fa[y] = x;      // 小根挂到大根下（按大小合并，树高 O(log n)）
        sz[x] += sz[y]; // 更新合并后的集合大小
        return true;    // 可用于 Kruskal 统计合并次数
    }

    // x、y 是否在同一集合
    bool same(int x, int y)
    {
        return get(x) == get(y);
    }

    int size(int x) // x 所在连通块大小（取根后读 sz）
    {
        return sz[get(x)];
    }
};
```

树状数组 Fenwick.hpp

- **用法**：Fenwick<ll> fw(n)；add(x, v) 单点加；query(x) 前缀和 [1..x]；query(l, r) 区间和；select(k) 树上二分。
- **复杂度**：单次 O(log n)。
- **select**：返回满足"前缀和 <= k"的最大下标 x（配合"每个位置 add(pos, 1)"可查第 k 小：答案为 select(k - 1) + 1）。

```
template <typename T> struct Fenwick
{
    int n;
    vector<T> tree; // tree[i] 保存区间 (i-lowbit(i), i] 的区间和

    // 构造大小为 n 的树状数组（下标 1..n）
    Fenwick(int n) : n(n), tree(n + 1)
    {
    }

    // 单点加：位置 x 增加 v
    void add(int x, const T &v)
    {
        for (; x ≤ n; x += x & -x) // x += lowbit(x)：逐层跳到"管辖 x 的更高点"
            tree[x] += v;
    }

    // 前缀和 [1..x]
    T query(int x)
    {
        T ans{};                    // 值初始化为 T 的零元
        for (; x > 0; x -= x & -x) // x -= lowbit(x)：逐层剥掉最低二进制位
            ans += tree[x];
        return ans;
    }

    // 区间和 [l..r] = 前缀(r) - 前缀(l-1)
    T query(int l, int r)
    {
        return query(r) - query(l - 1);
    }

    // 树上二分：返回满足"前缀和 ≤ k"的最大 x（配合每位置 add(pos,1) 查第 k 小）
    int select(const T &k)
    {
        int x = 0;
        T cur{};
        for (int i = 1 << __lg(n); i; i >>= 1) // 从最高二进制位往下试跳
            if (x + i ≤ n && cur + tree[x + i] ≤ k)
                cur += tree[x += i]; // 前缀和仍 ≤ k，就跳过去
        return x;
    }
};
```

线段树（区间加 / 区间求和）SegmentTree.hpp

- **用法**：SegmentTree<ll> sg(n) 或 SegmentTree<ll> sg(arr)（arr 为 1-indexed、长度 n+1）；update(l, r, k) 区间加 k；query(l, r) 区间和；重建可调 build()。
- **复杂度**：单次 O(log n)。
- **注意**：**依赖骨架宏 ls / rs**；T 需支持 +、+=，且懒标记 0 表示无标记；区间长用 (r - l + 1) * k，注意溢出（建议 T = ll）。

```
template <class T> struct SegmentTree
{
    int n;
    vector<T> in;          // 原数组（1-indexed）
    vector<T> tree, tag;   // tree[p]: 节点值（区间和）；tag[p]: 区间加懒标记（0 表示无）

    SegmentTree(int n) : n(n), in(n + 1), tree(4 * n), tag(4 * n)
    {
    }

    SegmentTree(vector<T> &arr) : n(arr.size() - 1), in(arr), tree(4 * n), tag(4 * n)
    {
        build(); // 直接从数组建树
    }

    // 由两个儿子合并出父亲（此处为区间和：左 + 右）
    void push_up(int p)
    {
        tree[p] = tree[ls(p)] + tree[rs(p)];
    }

    void build(int p, int l, int r) // 递归建树：节点 p 覆盖 [l, r]
    {
        if (l == r) // 叶子：直接取原数组值
        {
            tree[p] = in[l];
            return;
        }
        int mid = l + r >> 1;
        build(ls(p), l, mid);
        build(rs(p), mid + 1, r);
        push_up(p);
    }

    // 给 p 覆盖的整段 [l, r] 整体加 k，并打懒标记（O(1)）
    void change(int p, int l, int r, T k)
    {
        tree[p] += (r - l + 1) * k; // 区间和增加 长度 * k
        tag[p] += k;                // 懒标记累加，留待访问儿子时再下传
    }

    // 把 p 的懒标记下传给孩子（保证访问孩子前其值正确）
    void push_down(int p, int l, int r)
    {
        if (tag[p]) // 有标记才下传
        {
            int mid = l + r >> 1;
            change(ls(p), l, mid, tag[p]);
            change(rs(p), mid + 1, r, tag[p]);
            tag[p] = 0; // 清空本节点标记
        }
    }

    // 递归修改：把 [ul, ur] 整体加 k
```

```
    void update(int ul, int ur, int p, int l, int r, T k)
    {
        if (ul ⩽ l && r ⩽ ur) // 当前区间被 [ul, ur] 完全覆盖
        {
            change(p, l, r, k);
            return;
        }
        push_down(p, l, r); // 未完全覆盖，先下传再分别递归左右
        int mid = l + r >> 1;
        if (ul ⩽ mid)
            update(ul, ur, ls(p), l, mid, k);
        if (ur > mid)
            update(ul, ur, rs(p), mid + 1, r, k);
        push_up(p);
    }

    // 递归查询 [ql, qr] 的和
    T query(int ql, int qr, int p, int l, int r)
    {
        if (ql ⩽ l && r ⩽ qr) // 完全覆盖，直接返回节点值
            return tree[p];
        push_down(p, l, r);
        T ans = 0;
        int mid = l + r >> 1;
        if (ql ⩽ mid)
            ans += query(ql, qr, ls(p), l, mid);
        if (qr > mid)
            ans += query(ql, qr, rs(p), mid + 1, r);
        return ans;
    }

    // ---- 对外接口（以整棵树 [1, n] 为范围）----
    void build()
    {
        build(1, 1, n);
    }

    void update(int ul, int ur, T k) // [ul, ur] 区间加 k
    {
        update(ul, ur, 1, 1, n, k);
    }

    T query(int ql, int qr) // 查询 [ql, qr] 区间和
    {
        return query(ql, qr, 1, 1, n);
    }
};
```

ST 表（稀疏表） ST.hpp

- **用法**：`ST st(n)` 或 `ST st(arr)`（1-indexed、长度 `n+1`）；先 `init()`，再 `query(l, r)`。
- **复杂度**：预处理 `O(n log n)`，查询 `O(1)`。
- **注意**：仅适用于**可重复结合**（幂等）运算，默认取 `max`，可传自定义二元函数（如 `gcd`）；只支持 `int`。

```
struct ST
{
    const int n, len;                    // n: 元素个数; len = floor(log2 n)+1 层
    vector<int> in;                      // 原数组 (1-indexed)
    vector<vector<int>> st;              // st[i][j]: 区间 [j, j+2^i-1] 的答案
    function<int(int, int)> Max;         // 可重复结合的合并函数 (默认取 max)

    ST(
        int n, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(n), len(__lg(n) + 1), in(n + 1), Max(Max) // len 层足够覆盖任意长度
    {
        st.resize(len, vector<int>(n + 1));
    }

    ST(
        vector<int> &arr, function<int(int, int)> Max = [](int a, int b) { return max(a, b); })
        : n(arr.size() - 1), len(__lg(n) + 1), in(arr), Max(Max)
    {
        st.resize(len, vector<int>(n + 1));
    }

    // 预处理: 倍增递推, st[i][j] = Max(前半段, 后半段)
    void init()
    {
        for (int i = 1; i <= n; ++i)
            st[0][i] = in[i]; // 第 0 层: 长度为 1 的区间
        for (int i = 1; i < len; ++i)
            for (int j = 1; j + (1 << i) - 1 <= n; ++j)
                st[i][j] = Max(st[i - 1][j], st[i - 1][j + (1 << i - 1)]); // 两段 2^(i-1) 拼成 2^i
    }

    // O(1) 查询 [l, r]: 用两段长度 2^k 的区间覆盖 (可重叠, 故运算需幂等)
    int query(int l, int r)
    {
        int k = __lg(r - l + 1); // 不超过区间长度的最大 2 的幂
        return Max(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

整数位封装 Bit.hpp

- **用法**：bit<unsigned long long> b(x)；把整数的位运算封装成成员函数 / 运算符，可读性更好。
- **注意**：位下标从 LSB（第 0 位）开始；T 需为无符号整型（unsigned / unsigned long long 等），避免右移补符号位。小规模（位数少）直接裸位运算更快。

```
template <class T> struct bit // T 需为无符号整型（如 unsigned long long）
{
    T value;                                                    // 被封装的整数
    bit(T x = 0) : value(x) {}                                  // 构造（默认 0）
    // ---- 单比特操作（位下标从 LSB 第 0 位起）----
    bool operator[](int x) const { return value >> x & 1; }  // 读第 x 位
    void set(int x) { value ⊨ T(1) << x; }                      // 置第 x 位为 1
    void reset(int x) { value &= ~(T(1) << x); }                // 置第 x 位为 0
    void flip(int x) { value ^= T(1) << x; }                    // 翻转第 x 位
    // ---- 移位（返回新 bit，不修改自身）----
    bit operator<<(int x) const { return bit(value << x); }
    bit operator>>(int x) const { return bit(value >> x); }
    bit &operator⇐(int x) { value ⇐ x; return *this; }
    bit &operator⇒(int x) { value ⇒ x; return *this; }
    // ---- 按位运算 ----
    bit operator&(const bit &o) const { return bit(value & o.value); }
    bit operator|(const bit &o) const { return bit(value | o.value); }
    bit operator^(const bit &o) const { return bit(value ^ o.value); }
    bit &operator&=(const bit &o) { value &= o.value; return *this; }
    bit &operator⊨(const bit &o) { value ⊨ o.value; return *this; }
    bit &operator^=(const bit &o) { value ^= o.value; return *this; }
    bit operator~() const { return bit(~value); }               // 按位取反
    // ---- 比较 ----
    bool operator⩵(const bit &o) const { return value ⩵ o.value; }
    bool operator≠(const bit &o) const { return value ≠ o.value; }
    bool operator<(const bit &o) const { return value < o.value; }
    bool operator>(const bit &o) const { return value > o.value; }
    bool operator⩽(const bit &o) const { return value ⩽ o.value; }
    bool operator⩾(const bit &o) const { return value ⩾ o.value; }
};
```

FHQ Treap（数组版，按值 / 按大小分裂） Treap.hpp

- **用法**：Treap<int> tp(节点数上限)；insert(x) 插入；erase(x) 删除一个 x；rank(x) 比 x 小的元素个数 + 1；val(k) 第 k 小；pre(x) 前驱（< x 的最大值）；back(x) 后继（> x 的最小值）；count(x) x 的个数；size() 元素总数。
- **复杂度**：单次期望 O(log n)。
- **注意**：构造参数为**节点数上限**（内部开 n+1，每次 insert 新建一个节点）；split_val 的 eq 参数控制 == k 归左还是归右；指针版见 TreapPtr。

```
template <class T> struct Treap
{
    struct Node
    {
        T val;
        int key, siz, ch[2]; // key: 随机优先级（key 小者作根）；siz: 子树大小；ch[0/1]: 左右儿子
        void clear()
        {
            ch[0] = ch[1] = siz = key = 0;
            val = T{};
        }
    };

    int cnt, root;
    vector<Node> t;
    mt19937_64 gen;

    Treap(int n) : cnt(0), root(0), t(n + 1), gen(chrono::steady_clock::now().time_since_epoch().count
())
    {
    }

    int rnd()
    {
        return int(gen());
    }

    // 用儿子更新自己的子树大小
    void update(int now)
    {
        t[now].siz = t[t[now].ch[0]].siz + t[t[now].ch[1]].siz + 1;
    }

    // 合并两棵树（要求 u 中所有值 ≤ v 中所有值），按随机优先级 key 确定新根（key 小者作根）
    int merge(int u, int v)
    {
        if (!u || !v)
            return u | v;             // 有一棵为空：返回另一棵
        if (t[u].key < t[v].key)      // u 优先级更高，u 作根
        {
            t[u].ch[1] = merge(t[u].ch[1], v); // v 接到 u 右子树
            update(u);
            return u;
        }
        else                          // v 优先级更高，v 作根
        {
            t[v].ch[0] = merge(u, t[v].ch[0]); // u 接到 v 左子树
            update(v);
            return v;
        }
    }

    // 按值分裂：值 < k（eq 时含 == k）的节点进 x，其余进 y
    void split_val(int now, const T &k, int &x, int &y, bool eq = true)
```

```
{
    if (!now)
        return (void)(x = y = 0);
    if (t[now].val < k || (eq && t[now].val == k))
        x = now, split_val(t[now].ch[1], k, t[now].ch[1], y, eq); // now 归 x，继续分右子树
    else
        y = now, split_val(t[now].ch[0], k, x, t[now].ch[0], eq);  // now 归 y，继续分左子树
    update(now); // 回溯时更新子树大小
}

// 按名次分裂：前 k 小（含）进 x，其余进 y
void split_k(int now, int k, int &x, int &y)
{
    if (!now)
        return (void)(x = y = 0);
    if (t[t[now].ch[0]].siz < k) // 左子树不够 k 个，now 及左子树归 x
        x = now, split_k(t[now].ch[1], k - t[t[now].ch[0]].siz - 1, t[now].ch[1], y);
    else                         // 左子树已够 k 个，now 归 y
        y = now, split_k(t[now].ch[0], k, x, t[now].ch[0]);
    update(now); // 回溯时更新子树大小
}

// 取第 k 小（并合并回 now）
T kth(int &now, int k)
{
    int a, b, c, d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = t[c].val;
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 新建节点，按值分裂后插入中间
{
    int u = ++cnt, a, b;
    t[u].val = x, t[u].key = rnd(), t[u].siz = 1;
    split_val(root, x, a, b);
    root = merge(merge(a, u), b);
}

void erase(const T &x) // 删除一个 x（不存在则无操作）
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_k(b, 1, c, d);             // c: 值为 x 的一个节点
    if (c && t[c].val == x)
        t[c].clear(), root = merge(a, d);
    else
        root = merge(a, merge(c, d));
}

int size() // 元素总数
{
    return t[root].siz;
}

int count(const T &x) // x 出现次数
{
    int a, b, c, d;
    split_val(root, x, a, b, false); // a: < x
    split_val(b, x, c, d, true);     // c: == x
    int res = t[c].siz;
    root = merge(a, merge(c, d));
```

```
        return res;
    }

    int rank(const T &x) // x 的排名（比它小的个数 + 1）
    {
        int a, b;
        split_val(root, x, a, b, false);
        int res = t[a].siz + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱（< x 的最大值）
    {
        int a, b;
        split_val(root, x, a, b, false);
        T res = kth(a, t[a].siz);
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继（> x 的最小值）
    {
        int a, b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};
```

FHQ Treap（指针版，免预估节点数）`TreapPtr.hpp`

- **用法**：与 `Treap.hpp` 完全一致的 API；`TreapPtr<int> tp;` 直接用，无需估计节点数上限。
- **实现**：内存池 `pool` 统一管理新节点，删除的节点进回收站 `trash` 复用，析构时统一释放。
- **注意**：禁用了拷贝（避免双重释放）；`kth` 对空树返回 `T{}`。

```
template <class T> struct TreapPtr
{
    struct Node
    {
        T val;
        int key, siz;
        Node *ch[2];
        Node(const T &v = T{}, int k = 0) : val(v), key(k), siz(1)
        {
            ch[0] = ch[1] = nullptr;
        }
    };

    Node *root;
    vector<Node *> pool, trash; // 内存池 / 回收站
    mt19937_64 gen;

    TreapPtr() : root(nullptr), gen(chrono::steady_clock::now().time_since_epoch().count()) {}
    TreapPtr(const TreapPtr &) = delete;            // 禁止拷贝
    TreapPtr &operator=(const TreapPtr &) = delete;
    ~TreapPtr() // 析构统一释放内存池
    {
        for (Node *p : pool)
            delete p;
    }

    int rnd()
    {
        return int(gen());
    }

    int sz(Node *p) // 空节点大小为 0
    {
        return p ? p->siz : 0;
    }

    // 取新节点：优先复用回收站
    Node *newNode(const T &v)
    {
        Node *p;
        if (!trash.empty())
        {
            p = trash.back(), trash.pop_back();
            p->val = v, p->key = rnd(), p->siz = 1;
            p->ch[0] = p->ch[1] = nullptr;
        }
        else
            pool.push_back(p = new Node(v, rnd()));
        return p;
    }

    void update(Node *now)
    {
        now->siz = sz(now->ch[0]) + sz(now->ch[1]) + 1;
    }
```

```
Node *merge(Node *u, Node *v) // 合并（u 中所有值 ≤ v），按 key 小者作根
{
    if (!u || !v)
        return u ? u : v; // 有一棵为空：返回另一棵
    if (u→key < v→key)  // u 优先级更高，u 作根
    {
        u→ch[1] = merge(u→ch[1], v); // v 接到 u 右子树
        update(u);
        return u;
    }
    else                  // v 优先级更高，v 作根
    {
        v→ch[0] = merge(u, v→ch[0]); // u 接到 v 左子树
        update(v);
        return v;
    }
}

void split_val(Node *now, const T &k, Node *&x, Node *&y, bool eq = true) // 按值分裂
{
    if (!now)
        return (void)(x = y = nullptr);
    if (now→val < k || (eq && now→val == k))
        x = now, split_val(now→ch[1], k, now→ch[1], y, eq); // now 归 x，继续分右子树
    else
        y = now, split_val(now→ch[0], k, x, now→ch[0], eq);  // now 归 y，继续分左子树
    update(now);
}

void split_k(Node *now, int k, Node *&x, Node *&y) // 按名次分裂：前 k 小进 x，其余进 y
{
    if (!now)
        return (void)(x = y = nullptr);
    if (sz(now→ch[0]) < k) // 左子树不够 k 个，now 及左子树归 x
        x = now, split_k(now→ch[1], k - sz(now→ch[0]) - 1, now→ch[1], y);
    else                   // 左子树已够 k 个，now 归 y
        y = now, split_k(now→ch[0], k, x, now→ch[0]);
    update(now);
}

T kth(Node *&now, int k) // 第 k 小
{
    Node *a, *b, *c, *d;
    split_k(now, k - 1, a, b);
    split_k(b, 1, c, d);
    T res = c ? c→val : T{};
    now = merge(a, merge(c, d));
    return res;
}

void insert(const T &x) // 插入
{
    Node *a, *b;
    split_val(root, x, a, b);
    root = merge(merge(a, newNode(x)), b);
}

void erase(const T &x) // 删除一个 x，节点进回收站
{
    Node *a, *b, *c, *d;
    split_val(root, x, a, b, false);
    split_k(b, 1, c, d);
    if (c && c→val == x)
        trash.push_back(c), root = merge(a, d);
```

```
        else
            root = merge(a, merge(c, d));
    }

    int size() // 元素总数
    {
        return sz(root);
    }

    int count(const T &x) // x 出现次数
    {
        Node *a, *b, *c, *d;
        split_val(root, x, a, b, false);
        split_val(b, x, c, d, true);
        int res = sz(c);
        root = merge(a, merge(c, d));
        return res;
    }

    int rank(const T &x) // x 的排名
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        int res = sz(a) + 1;
        root = merge(a, b);
        return res;
    }

    T val(int k) // 第 k 小
    {
        return kth(root, k);
    }

    T pre(const T &x) // 前驱
    {
        Node *a, *b;
        split_val(root, x, a, b, false);
        T res = kth(a, sz(a));
        root = merge(a, b);
        return res;
    }

    T back(const T &x) // 后继
    {
        Node *a, *b;
        split_val(root, x, a, b);
        T res = kth(b, 1);
        root = merge(a, b);
        return res;
    }
};
```

莫队 Mo.hpp（片段式）

> **片段式**（同 Math/Exgcd.hpp、Graph/Hungarian.hpp）：文件里是若干可直接粘贴的骨架，**不能作为 #include 单独编译**；四段为互斥变体，每次只取其中一段。 约定 **1-indexed**；依赖 ihxnan.hpp 的别名 ti/vi/vl 与分块宏 bi(x)（并要求作用域内存在 int blen，用到 bl/br 时还需 int n）。 共同骨架：查询读进 vector<ti> qry（(l, r, id)）→ 按 bi(l) 分块 + **奇偶优化**排序 → 空窗口 left=1, right=0 四向移动调用 ins/del → ans[id] = 当前贡献。

普通莫队（P2709 / P3901 / P1494）

- **用法**：int blen = sqrt(n) 后排序、开空窗口；只需按题目改写 ins / del 两个 lambda（增删一个下标对答案的贡献）。
- **复杂度**：块长 $\sqrt{n}$，总复杂度 $O((n+m)\sqrt{n})$。
- **注意**：ins / del 以 P2709 的"同色对数"为例；P3901 只需改成维护"种类数"，P1494 改成维护 A, B。

```
int blen = sqrt(n); // 块长一般取 sqrt(n)
vector<ti> qry(m);  // (l, r, id), 1-indexed
int dn = 0;
for (auto &[l, r, id] : qry)
    cin >> l >> r, id = dn++;

sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, id1] = x;
    auto &[l2, r2, id2] = y;
    if (bi(l1) != bi(l2))
        return bi(l1) < bi(l2);
    return bi(l1) & 1 ? r1 > r2 : r1 < r2; // 奇偶优化：块内 r 交替方向
});

vl ans(m);
int left = 1, right = 0; // 空窗口 [1, 0]

// ---- 改这两个 lambda ----
vi cnt(k + 1);
ll res = 0;
auto ins = [&](int x) -> void { res += 2 * ++cnt[arr[x]] - 1; };
auto del = [&](int x) -> void { res -= 2 * cnt[arr[x]]-- - 1; };
// ------------------------

for (auto &[l, r, id] : qry)
{
    while (left < l)
        del(left++);
    while (left > l)
        ins(--left);
    while (right < r)
        ins(++right);
    while (right > r)
        del(right--);
    ans[id] = res;
}
for (auto &p : ans)
    cout << p << endl;
```

莫队 + 值域分块求 mex（P4137）

- **用法**：在普通莫队基础上追加值域块计数 tag[]；ins/del 同时维护 cnt[] 与 tag[bi(值)]（只统计 arr[x] <= n 的值）；回答时先跳到第一个未满的块，再块内线性找第一个未出现的值。
- **复杂度**：查询 $O(\sqrt{n})$ 求 mex，总 $O((n+m)\sqrt{n})$。

- **注意**：值域含 `0`，块长取 `max(2, (int)sqrt(n))` 才能保证 `bi(0) = 0`（值 `0` 归入第 `0` 块）；第 `0` 块比普通块多含一个值 `0`（`+ !idx`）。

```
int blen = max(2, (int)sqrt(n)); // 值域分块需 blen≥2 才能保证 bi(0)=0
int bnum = (n + blen - 1) / blen; // 值域块数

// ... 读入 + 排序 + ins/del 同普通莫队 ...
vi tag(bnum), ans(m), cnt(n + 1);
auto ins = [&](int x) → void {
    if (arr[x] ⩽ n && ++cnt[arr[x]] == 1)
        ++tag[bi(arr[x])];
};
auto del = [&](int x) → void {
    if (arr[x] ⩽ n && !--cnt[arr[x]])
        --tag[bi(arr[x])];
};

// ... 窗口移动后求 mex ...
int idx = 0; // 第一个未满的块（第 0 块含值 0，多一个）
while (tag[idx] == br(idx) - bl(idx) + 1 + !idx)
    ++idx;
idx = bl(idx) - !idx; // 定位到该块左端（第 0 块从 0 开始）
while (cnt[idx])      // 块内找第一个未出现的值
    ++idx;
ans[id] = idx;
```

带修莫队（P1903）

- **用法**：查询是三维 `(l, r, 时间 t)`；修改存 `(pos, after, before)`，用 `front/back` 沿时间轴应用/撤销；块长取 `n^(2/3)`，排序三级（`bi(l)` → 块内 `r` 奇偶 → `bi(l)+bi(r)` 奇偶决定 `t` 方向）。
- **复杂度**：块长 `n^(2/3)`，总 `O(n^(5/3))`。
- **注意**：`front` / `back` 仅当修改落点 `pos` 在当前窗口 `[left, right]` 内时才需改动贡献。

```
int blen = pow(n, 0.66);               // 带修块长 ≈ n^(2/3)
vector<tuple<int, int, int, int>> qry; // (l, r, t, id)
vector<ti> func;                       // (pos, after, before)
int t = 0, dn = 0;
vi tmp = arr; // 记录每次修改的 before
for (int i = 0, l, r; i < m; ++i)
{
    char op;
    cin >> op >> l >> r;
    if (op == 'Q')
        qry.emplace_back(l, r, t, dn++);
    else
        func.emplace_back(l, r, tmp[l]), tmp[l] = r, ++t;
}
sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, t1, id1] = x;
    auto &[l2, r2, t2, id2] = y;
    if (bi(l1) != bi(l2))
        return bi(l1) < bi(l2);
    if (bi(r1) != bi(r2))
        return bi(l1) & 1 ? r1 > r2 : r1 < r2;
    return bi(l1) + bi(r1) & 1 ? t1 > t2 : t1 < t2;
});

// ... ins/del 同普通莫队 ...
auto front = [&](int x) → void { // 时间前进：应用第 x 次修改
    auto &[pos, after, before] = func[x];
    arr[pos] = after;
    if (left ⩽ pos && pos ⩽ right)
```

```
        sum -= !--cnt[before], sum += ++cnt[after] == 1;
};
auto back = [&](int x) -> void { // 时间回退：撤销第 x 次修改
    auto &[pos, after, before] = func[x];
    arr[pos] = before;
    if (left <= pos && pos <= right)
        sum -= !--cnt[after], sum += ++cnt[before] == 1;
};

// ... 四向移动 + 时间轴对齐 ...
while (time < t)
    front(time++);
while (time > t)
    back(--time);
ans[id] = sum;
```

回滚莫队（P14420）

- **用法**：适用于"只有加入好维护、删除难维护"（如区间 max）的情形；右端点在块内只增，左端点临时加入后整体回滚。计数用 unordered_map（值域大）。
- **复杂度**：块长 $\sqrt{n}$，总 $O(n\sqrt{n})$。
- **注意**：del 只回退计数、**不维护 cur**，故进入新块重置窗口后需显式 cur = 0；左端回退目标是"上一块右端之后"，用 br(block - 1) + 1——末块不满时 bl(block) 会越过 n 造成越界。

```
int blen = sqrt(n);
vector<ti> qry(q); // (l, r, id)
int dn = 0;
for (auto &[l, r, id] : qry)
    cin >> l >> r, id = dn++;
sort(qry.begin(), qry.end(), [&](auto &x, auto &y) {
    auto &[l1, r1, id1] = x;
    auto &[l2, r2, id2] = y;
    if (bi(l1) != bi(l2))
        return l1 < l2;
    return r1 < r2; // 同块内 r 升序
});

vl ans(q);
int block = -1;
ll cur = 0, back = 0;
int left = 1, right = 0;
unordered_map<int, int> cnt;
auto ins = [&](int x) -> void { cur = max(cur, arr[x] * ++cnt[arr[x]]); };
auto del = [&](int x) -> void { --cnt[arr[x]]; }; // 只回退计数，不动 cur

for (auto &[l, r, id] : qry)
{
    if (block <= bi(l)) // 进入新块：重置为该块右侧的空窗口
    {
        block = bi(l) + 1;
        while (left < br(block - 1) + 1) // 末块不满时 bl(block) 越界，用 br+1
            del(left++);
        while (right < br(block - 1))
            ins(++right);
        while (right > br(block - 1))
            del(right--);
        cur = 0;
    }
    if (bi(l) == bi(r)) // 同块：暴力扫
    {
        ll res = 0;
        for (int i = l; i <= r; ++i)
```

```
            res = max(res, arr[i] * ++cnt[arr[i]]);
        for (int i = l; i <= r; ++i)
            --cnt[arr[i]];
        ans[id] = res;
    }
    else // 跨块: 右端只增, 左端临时加入后回滚
    {
        while (right < r)
            ins(++right);
        back = cur;
        while (left > l)
            ins(--left);
        ans[id] = cur;
        while (left < br(block - 1) + 1)
            del(left++);
        cur = back;
    }
}
for (auto &p : ans)
    cout << p << endl;
```

图论 Graph/

> 约定：**除 Hungarian（1-indexed 片段）与 HLD（1-indexed，根默认 1）外，均 0-indexed**（点号 0..n-1）。 依赖 Template/ihxnan.hpp 的类型别名（ll、vi、pii 等）。 **冲突提醒：** MaxFlow.hpp 与 MaxAssignment.hpp 都在文件作用域定义了 constexpr int inf，同一个文件只能粘贴其一（或改名）。

最大流（Dinic） MaxFlow.hpp

- **用法**：MaxFlow<ll> mf(n)；addEdge(u, v, c) 加有向边（自动建反向边）；flow(s, t) 返回最大流；minCut() 求完流后返回每个点是否在 S 侧；edges() 取每条边流量明细。
- **复杂度**：O(n^2 m)，一般图远快于此；单位容量图 O(E√E)。
- **注意**：**0-indexed**；文件作用域有 constexpr int inf = 1E9，与 MaxAssignment.hpp 的 inf 冲突（同文件只能粘贴其一，或改名）。

```
constexpr int inf = 1E9; // 全局无穷大（与 MaxAssignment.hpp 的 inf 冲突，勿同时粘贴）
template <class T>
struct MaxFlow { // 最大流（Dinic）：0-indexed
    struct _Edge {
        int to;   // 出边指向的点
        T cap;    // 剩余容量
        _Edge(int to, T cap) : to(to), cap(cap) {}
    };

    int n;
    std::vector<_Edge> e;             // 边表，成对存储：i 与 i^1 互为反向边
    std::vector<std::vector<int>> g;  // 邻接表存边下标
    std::vector<int> cur, h;          // cur: 当前弧优化；h: BFS 分层

    MaxFlow() {}
    MaxFlow(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        e.clear();
        g.assign(n, {});
        cur.resize(n);
        h.resize(n);
    }

    // BFS 分层，标到 t 即可提前退出
    bool bfs(int s, int t) {
        h.assign(n, -1);
        std::queue<int> que;
        h[s] = 0;
        que.push(s);
        while (!que.empty()) {
            const int u = que.front();
            que.pop();
            for (int i : g[u]) {
                auto [v, c] = e[i];
                if (c > 0 && h[v] == -1) { // 有剩余容量且未分层
                    h[v] = h[u] + 1;
                    if (v == t) {
                        return true;
                    }
                    que.push(v);
                }
            }
        }
        return false;
    }

    // 多路增广：从 u 出发最多推 f 的流量
    T dfs(int u, int t, T f) {
        if (u == t) {
            return f;
```

```
        }
        auto r = f;
        for (int &i = cur[u]; i < int(g[u].size()); ++i) { // 当前弧优化
            const int j = g[u][i];
            auto [v, c] = e[j];
            if (c > 0 && h[v] == h[u] + 1) { // 只走相邻层
                auto a = dfs(v, t, std::min(r, c));
                e[j].cap -= a;     // 正向边扣容量
                e[j ^ 1].cap += a; // 反向边加容量
                r -= a;
                if (r == 0) {
                    return f;
                }
            }
        }
        return f - r;
    }

    // 加一条 u→v 容量 c 的有向边，同时自动建反向边（u←v 容量 0）
    void addEdge(int u, int v, T c)
    {
        g[u].push_back(e.size()); // u 记录正向边下标
        e.emplace_back(v, c);     // 正向边（偶数下标 i）
        g[v].push_back(e.size());
        e.emplace_back(u, 0);     // 反向边（下标 i^1）
    }

    T flow(int s, int t) // 求 s → t 最大流
    {
        T ans = 0;
        while (bfs(s, t)) {
            cur.assign(n, 0);
            ans += dfs(s, t, std::numeric_limits<T>::max());
        }
        return ans;
    }

    // 最小割划分：返回 c[i] = true 表示 i 在 S 侧
    std::vector<bool> minCut() {
        std::vector<bool> c(n);
        for (int i = 0; i < n; i++) {
            c[i] = (h[i] != -1); // 残量网络仍可达的即 S 侧
        }
        return c;
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T flow;
    };

    // 取所有边的 {from, to, 容量, 流量}
    std::vector<Edge> edges() {
        std::vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
```

```
        return a;
    }
};
```

费用流（SPFA 朴素版，默认） MinCostFlow.hpp

- **用法**：`MinCostFlow<ll> mcf(n)`；`add(u, v, cap, cost)`；`flow(s, t)` 返回 `{最大流, 最小费用}`；`flow(s, t, k)` 限制流量上限 `k`；`edges()` 取边明细。
- **复杂度**：SPFA 增广，够用于多数题目。
- **注意**：**0-indexed**；可直接加**负费用边**（要求图中无负费用环）；模板参数 `T` 同时决定容量与费用类型。

```
template <class T> struct MinCostFlow // 最小费用最大流（SPFA 朴素版）：0-indexed
{
    struct _Edge
    {
        int to;   // 出边指向的点
        T cap;    // 剩余容量
        T cost;   // 单位费用（反向边为 -cost）

        _Edge(int _to, T _cap, T _cost) : to(_to), cap(_cap), cost(_cost)
        {
        }
    };

    struct Edge // 对外边明细
    {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    int n;
    vector<_Edge> e;  // 边表，成对存储（i 与 i^1 互为反向边）
    vector<vector<int>> g;
    vector<T> dis;
    vector<int> pre;  // 增广路前驱边

    MinCostFlow()
    {
    }

    MinCostFlow(int n_) : n(n_), g(n_)
    {
    }

    void init(int n_)
    {
        n = n_;
        e.clear();
        g.assign(n, {});
    }

    void add(int u, int v, T cap, T cost) // 正向容量 cap、费用 cost；反向容量 0、费用 -cost
    {
        g[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    // SPFA 找费用最短增广路
    bool spfa(int s, int t)
    {
        dis.assign(n, numeric_limits<T>::max());
```

```
        pre.assign(n, -1);
        vector<bool> inq(n, false);
        queue<int> que;
        dis[s] = 0;
        que.push(s);
        inq[s] = true;
        while (!que.empty())
        {
            int u = que.front();
            que.pop();
            inq[u] = false;
            for (int i : g[u])
            {
                int v = e[i].to;
                T cap = e[i].cap;
                T cost = e[i].cost;
                if (cap > 0 && dis[v] > dis[u] + cost) // 松弛
                {
                    dis[v] = dis[u] + cost;
                    pre[v] = i;
                    if (!inq[v])
                    {
                        inq[v] = true;
                        que.push(v);
                    }
                }
            }
        }
        return dis[t] != numeric_limits<T>::max(); // t 可达
    }

    // k 为流量上限，返回 {流量, 费用}
    pair<T, T> flow(int s, int t, int k = iINF)
    {
        T flow = 0;
        T cost = 0;
        while (spfa(s, t) && flow < k)
        {
            T aug = numeric_limits<T>::max();      // 增广量 = 路径最小残量
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
                aug = min(aug, e[pre[i]].cap);
            for (int i = t; i != s; i = e[pre[i] ^ 1].to)
            {
                e[pre[i]].cap -= aug;   // 正向扣
                e[pre[i] ^ 1].cap += aug; // 反向加
            }
            flow += aug;
            cost += aug * dis[t]; // 路径费用 = dis[t]
        }
        return make_pair(flow, cost);
    }

    vector<Edge> edges() // 取所有边的 {from, to, 容量, 费用, 流量}
    {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2)
        {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
```

```
        }
        return a;
    }
};
```

费用流（Dijkstra + 势优化，被卡时间再换） MinCostFlowDijkstra.hpp

- **用法**：与 SPFA 版同名同接口：`add(u, v, cap, cost)`、`flow(s, t)` 返回 `{最大流, 最小费用}`。
- **复杂度**：每轮增广 $O(m \log n)$，总 $O(\text{flow} \cdot m \log n)$。
- **注意**：**0-indexed**；要求**初始边费用非负**（用Johnson 势 `h` 消除负权后跑 Dijkstra）。

```
template <class T>
struct MinCostFlow {
    struct _Edge {
        int to;
        T cap;
        T cost;
        _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
    };
    int n;
    vector<_Edge> e;  // 成对存储，i 与 i^1 互为反向边
    vector<vector<int>> g;
    vector<T> h, dis; // h: 势（Johnson）；dis: 缩权后距离
    vector<int> pre;

    bool dijkstra(int s, int t) {
        dis.assign(n, numeric_limits<T>::max());
        pre.assign(n, -1);
        priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>> que;
        dis[s] = 0;
        que.emplace(0, s);
        while (!que.empty()) {
            T d = que.top().first;
            int u = que.top().second;
            que.pop();
            if (dis[u] != d) { // 过期堆元素
                continue;
            }
            for (int i : g[u]) {
                int v = e[i].to;
                T cap = e[i].cap;
                T cost = e[i].cost;
                // 缩权：cost + h[u] - h[v] ≥ 0
                if (cap > 0 && dis[v] > d + h[u] - h[v] + cost) {
                    dis[v] = d + h[u] - h[v] + cost;
                    pre[v] = i;
                    que.emplace(dis[v], v);
                }
            }
        }
        return dis[t] != numeric_limits<T>::max();
    }

    MinCostFlow() {}
    MinCostFlow(int n_) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        e.clear();
        g.assign(n, {});
    }

    void add(int u, int v, T cap, T cost) // 正向容量 cap，反向容量 0、费用 -cost
    {
        g[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
```

```
        g[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    pair<T, T> flow(int s, int t) // 返回 {最大流, 最小费用}
    {
        T flow = 0;
        T cost = 0;
        h.assign(n, 0);
        while (dijkstra(s, t)) {
            for (int i = 0; i < n; ++i) {
                h[i] += dis[i]; // 更新势: h[i] += 真实距离
            }
            T aug = numeric_limits<int>::max(); // 增广量 (注意此处按 int 上限取)
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                aug = min(aug, e[pre[i]].cap);
            }
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                e[pre[i]].cap -= aug;
                e[pre[i] ^ 1].cap += aug;
            }
            flow += aug;
            cost += aug * h[t]; // h[t] 此时即 s→t 真实最短费用
        }
        return make_pair(flow, cost);
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    vector<Edge> edges() // 边明细
    {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

强连通分量（Tarjan） SCC.hpp

- **用法**：SCC scc(n)；add(u, v) 有向边；work() 返回 bel，bel[i] 为 i 所在 SCC 编号。
- **复杂度**：O(n + m)。
- **注意**：**0-indexed**；编号为**逆拓扑序**（对边 u->v 有 bel[v] <= bel[u]）；递归实现，深图注意栈空间。

```
struct SCC { // 强连通分量（Tarjan）：0-indexed
    int n;
    vector<vector<int>> adj;   // 邻接表
    vector<int> stk;           // Tarjan 栈（存当前 SCC 候选点）
    vector<int> dfn, low, bel; // dfn: 时间戳；low: 追溯值；bel: 所属 SCC 编号
    int cur, cnt;              // cur: 时间戳计数；cnt: SCC 计数

    SCC() {}
    SCC(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void add(int u, int v) { // 有向边
        adj[u].push_back(v);
    }

    void dfs(int x) {
        dfn[x] = low[x] = cur++; // 时间戳
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (dfn[y] == -1) { // 树边
                dfs(y);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1) { // 返祖边（y 还在栈中）
                low[x] = min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) { // x 是 SCC 根，弹出栈内成一分量
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
        }
    }

    vector<int> work() // 遍历所有点，返回 bel
    {
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
                dfs(i);
            }
```

```
        }
        return bel;
    }
};
```

边双连通分量（EBCC）`EBCC.hpp`

- **用法**：`EBCC ebcc(n)`；`add(u, v)` 无向边；`work()` 返回 `bel`（每个点的边双编号）；`compress()` 得到缩点后的信息。
- **复杂度**：`O(n + m log m)`（`set` 去重）。
- **注意**：**0-indexed**；`work()` 只从点 `0` 出发一次 DFS，**要求图连通**（不连通需对每个未访问点分别调用）；文件作用域有全局 `set<pii> E`（记录遍历中遇到的边，用于处理重边）。

```
set<pii> E; // 去重边集合（处理重边）

struct EBCC { // 边双连通分量：0-indexed
    int n;
    vector<vector<int>> adj;   // 邻接表
    vector<int> stk;           // Tarjan 栈
    vector<int> dfn, low, bel; // dfn: 时间戳；low: 追溯值；bel: 所属边双编号
    int cur, cnt;              // cur: 时间戳计数；cnt: 边双计数

    EBCC() {}
    EBCC(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        bel.assign(n, -1);
        stk.clear();
        cur = cnt = 0;
    }

    void add(int u, int v) { // 无向边
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int x, int p) {
        dfn[x] = low[x] = cur++;
        stk.push_back(x);

        for (auto y : adj[x]) {
            if (y == p) { // 只跳过一条父边，重边得以保留
                continue;
            }
            if (dfn[y] == -1) { // 树边
                E.emplace(x, y);
                dfs(y, x);
                low[x] = min(low[x], low[y]);
            } else if (bel[y] == -1 && dfn[y] < dfn[x]) { // 返祖边
                E.emplace(x, y);
                low[x] = min(low[x], dfn[y]);
            }
        }

        if (dfn[x] == low[x]) { // x 是边双根，弹出栈内成一个边双
            int y;
            do {
                y = stk.back();
                bel[y] = cnt;
                stk.pop_back();
            } while (y != x);
            cnt++;
```

```
        }
    }

    vector<int> work() // 返回 bel（从点 0 出发，图需连通）
    {
        dfs(0, -1);
        return bel;
    }

    // 缩点结果：n = 边双个数；edges 为边双间树边；siz 为各边双点数；cnte 为各边双内部边数
    struct Graph {
        int n;
        vector<pii> edges;
        vector<int> siz;
        vector<int> cnte;
    };
    Graph compress() {
        Graph g;
        g.n = cnt;
        g.siz.resize(cnt);
        g.cnte.resize(cnt);
        for (int i = 0; i < n; i++) {
            g.siz[bel[i]]++;
            for (auto j : adj[i]) {
                if (bel[i] < bel[j]) { // 跨边双：树边（只记一次方向）
                    g.edges.emplace_back(bel[i], bel[j]);
                } else if (i < j) { // 同边双内部边（无序只计一次）
                    g.cnte[bel[i]]++;
                }
            }
        }
        return g;
    }
};
```

圆方树（点双 → 块割树） BlockCutTree.hpp

- **用法**：`BlockCutTree bct(n)`；`add(u, v)` 无向边；`work()` 返回 `{方点个数, 树边集}`。
- **建树**：原点 `0..n-1`；每个点双（V-BCC）新建一个方点 `n+i`；返回的 `edges` 是方点与其中所有原点的连边。
- **复杂度**：`O(n + m)`。
- **注意**：**0-indexed**；递归实现。

```
struct BlockCutTree { // 圆方树（点双）：0-indexed，原点 0..n-1，方点 n+i
    int n;
    vector<vector<int>> adj;   // 原图邻接表
    vector<int> dfn, low, stk; // dfn: 时间戳；low: 追溯值；stk: 点栈
    int cnt, cur;              // cnt: 点双个数（方点编号）；cur: 时间戳计数
    vector<pii> edges;         // 圆方树的边（方点–原点）

    BlockCutTree() {}
    BlockCutTree(int n) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        adj.assign(n, {});
        dfn.assign(n, -1);
        low.resize(n);
        stk.clear();
        cnt = cur = 0;
        edges.clear();
    }

    void add(int u, int v) { // 无向边
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int x) {
        stk.push_back(x);
        dfn[x] = low[x] = cur++;

        for (auto y : adj[x]) {
            if (dfn[y] == -1) {
                dfs(y);
                low[x] = min(low[x], low[y]);
                if (low[y] == dfn[x]) { // y 的子树无法绕过 x：弹出一个点双
                    int v;
                    do {
                        v = stk.back();
                        stk.pop_back();
                        edges.emplace_back(n + cnt, v); // 方点连原点
                    } while (v != y);
                    edges.emplace_back(x, n + cnt); // 割点 x 也属于该点双
                    cnt++;
                }
            } else {
                low[x] = min(low[x], dfn[y]);
            }
        }
    }

    pair<int, vector<pii>> work() // 返回 {方点个数, 树边集}
    {
        for (int i = 0; i < n; i++) {
            if (dfn[i] == -1) {
```

```
                stk.clear();
                dfs(i);
            }
        }
        return {cnt, edges};
    }
};
```

2-SAT `TwoSat.hpp`

- **用法**：`TwoSat ts(n)`（变量 `0..n-1`）；`addClause(u, f, v, g)` 加约束 `(u==f) || (v==g)`（`f` / `g` 为期望布尔值）；`satisfiable()` 判可解；`answer()` 取方案。
- **复杂度**：`O(n + m)`。
- **注意**：变量 **0-indexed**；内部节点 `2i` 表示 `i` 取 `false`、`2i+1` 表示取 `true`；Tarjan 递归实现，规模大时注意栈空间。

```
struct TwoSat { // 2-SAT: 变量 0..n-1
    int n;                  // 变量个数
    vector<vector<int>> e;  // 蕴含图，节点 2i 表示 i=false、2i+1 表示 i=true
    vector<bool> ans;       // 每个变量的取值
    TwoSat(int n) : n(n), e(2 * n), ans(n) {}

    // 约束：(u=f) || (v=g)，即 !f→g、!g→f 两条蕴含边
    void addClause(int u, bool f, int v, bool g) {
        e[2 * u + !f].push_back(2 * v + g);
        e[2 * v + !g].push_back(2 * u + f);
    }

    bool satisfiable() {
        vector<int> id(2 * n, -1), dfn(2 * n, -1), low(2 * n, -1);
        vector<int> stk;
        int now = 0, cnt = 0;
        // 内嵌 Tarjan 求 SCC
        function<void(int)> tarjan = [&](int u) {
            stk.push_back(u);
            dfn[u] = low[u] = now++;
            for (auto v : e[u]) {
                if (dfn[v] == -1) {
                    tarjan(v);
                    low[u] = min(low[u], low[v]);
                } else if (id[v] == -1) {
                    low[u] = min(low[u], dfn[v]);
                }
            }
            if (dfn[u] == low[u]) {
                int v;
                do {
                    v = stk.back();
                    stk.pop_back();
                    id[v] = cnt;
                } while (v != u);
                ++cnt;
            }
        };
        for (int i = 0; i < 2 * n; ++i)
            if (dfn[i] == -1)
                tarjan(i);
        for (int i = 0; i < n; ++i) {
            if (id[2 * i] == id[2 * i + 1]) return false; // 同真同假：无解
            // Tarjan 逆拓扑序：编号大的分量先被选
            ans[i] = id[2 * i] > id[2 * i + 1];
        }
        return true;
    }

    vector<bool> answer() { return ans; }
};
```

二分图最大匹配（匈牙利） Hungarian.hpp

- **用法**：**代码片段**，整体粘贴进 solve() （[&] 捕获局部变量）。要求已有：左部到右部的邻接表 adj （**1-indexed**），全局 n 。
- **调用**：取消注释的循环，对左部点 1..m 逐个增广， cnt 即最大匹配数。
- **复杂度**： O(n · m) 。
- **注意**： match[p] 为右部点 p 当前匹配的左部点（ 0 表示未匹配，因此点号必须从 1 开始）。

```
vector<bool> visit(n + 1);
vector<int> match(n + 1);
auto dfs = [&](auto &&self, int x) → bool { // x: 左部点
    for (auto &p : adj[x])                  // p: 右部点
        if (!visit[p])
        {
            visit[p] = true;
            if (!match[p] || self(self, match[p])) // p 空闲，或其原匹配可换
            {
                match[p] = x;
                return true;
            }
        }
    return false;
};

// int cnt = 0;
// for (int i = 1; i ⩽ m; ++i)       // 枚举左部点增广
// {
//     fill(visit.begin(), visit.end(), false);
//     cnt += dfs(dfs, i);
// }
```

二分图最大权完美匹配（KM） MaxAssignment.hpp

- **用法**：`MaxAssignment<ll> km; ll ans = km.solve(nx, ny, a);` —— `a` 为 `nx × ny` 的非负权矩阵，要求 `nx <= ny`；`assignment()` 返回每个左部点匹配的右部点（`xy[i]`）；`weights()` 返回"恰好匹配前 k 个"的最大权前缀和数组。
- **复杂度**：`O(n^2 m)`。
- **注意**：**0-indexed**；文件作用域有 `constexpr int inf = 1E7`，与 `MaxFlow.hpp` 的 `inf` 冲突；权值须非负。

```
constexpr int inf = 1E7;
template <class T>
struct MaxAssignment {
    public:
        // 返回 nx 个左部点全部匹配的最大权值和
        T solve(int nx, int ny, vector<vector<T>> a) {
            assert(0 ⩽ nx && nx ⩽ ny);
            assert(int(a.size()) == nx);
            for (int i = 0; i < nx; ++i) {
                assert(int(a[i].size()) == ny);
                for (auto x : a[i])
                    assert(x ⩾ 0); // 权值非负
            }

            // 用 x 更新右部点的 slack（松弛量）与来源 slackx
            auto update = [&](int x) {
                for (int y = 0; y < ny; ++y) {
                    if (lx[x] + ly[y] - a[x][y] < slack[y]) {
                        slack[y] = lx[x] + ly[y] - a[x][y];
                        slackx[y] = x;
                    }
                }
            };

            costs.resize(nx + 1);
            costs[0] = 0;
            lx.assign(nx, numeric_limits<T>::max()); // 左部顶标
            ly.assign(ny, 0);                        // 右部顶标
            xy.assign(nx, -1);                       // 左 → 右匹配
            yx.assign(ny, -1);                       // 右 → 左匹配
            slackx.resize(ny);
            for (int cur = 0; cur < nx; ++cur) { // 逐个加入左部点
                queue<int> que;
                visx.assign(nx, false);
                visy.assign(ny, false);
                slack.assign(ny, numeric_limits<T>::max());
                p.assign(nx, -1); // 交错树上 x 的前驱

                for (int x = 0; x < nx; ++x) {
                    if (xy[x] == -1) { // 未匹配的 x 作为根
                        que.push(x);
                        visx[x] = true;
                        update(x);
                    }
                }

                int ex, ey;        // 找到的增广终点
                bool found = false;
                while (!found) {
                    // 1) 在相等子图内 BFS 找增广路
                    while (!que.empty() && !found) {
                        auto x = que.front();
                        que.pop();
                        for (int y = 0; y < ny; ++y) {
                            if (a[x][y] == lx[x] + ly[y] && !visy[y]) { // 相等边
```

```
                        if (yx[y] == -1) { // y 未匹配：增广成功
                            ex = x;
                            ey = y;
                            found = true;
                            break;
                        }
                        que.push(yx[y]); // 顺着匹配边走
                        p[yx[y]] = x;
                        visy[y] = visx[yx[y]] = true;
                        update(yx[y]);
                    }
                }
            }
            if (found)
                break;

            // 2) 相等子图内无增广路：调整顶标
            T delta = numeric_limits<T>::max();
            for (int y = 0; y < ny; ++y)
                if (!visy[y])
                    delta = min(delta, slack[y]);
            for (int x = 0; x < nx; ++x)
                if (visx[x])
                    lx[x] -= delta;
            for (int y = 0; y < ny; ++y) {
                if (visy[y]) {
                    ly[y] += delta;
                } else {
                    slack[y] -= delta;
                }
            }
            // 3) slack 减到 0 的边进入相等子图，继续增广
            for (int y = 0; y < ny; ++y) {
                if (!visy[y] && slack[y] == 0) {
                    if (yx[y] == -1) {
                        ex = slackx[y];
                        ey = y;
                        found = true;
                        break;
                    }
                    que.push(yx[y]);
                    p[yx[y]] = slackx[y];
                    visy[y] = visx[yx[y]] = true;
                    update(yx[y]);
                }
            }
        }

        // 沿交错路翻转匹配，顺便维护 costs 前缀
        costs[cur + 1] = costs[cur];
        for (int x = ex, y = ey, ty; x != -1; x = p[x], y = ty) {
            costs[cur + 1] += a[x][y];
            if (xy[x] != -1)
                costs[cur + 1] -= a[x][xy[x]]; // 减去旧匹配
            ty = xy[x];
            xy[x] = y;
            yx[y] = x;
        }
    }
    return costs[nx];
}
vector<int> assignment() { // 每个左部点的匹配对象
    return xy;
}
```

```
        pair<vector<T>, vector<T>> labels() { // 顶标
            return make_pair(lx, ly);
        }
        vector<T> weights() { // costs[k]: 只匹配前 k 个的最优权值和
            return costs;
        }
    private:
        vector<T> lx, ly, slack, costs;
        vector<int> xy, yx, p, slackx;
        vector<bool> visx, visy;
};
```

重链剖分（HLD，1-indexed）HLD.hpp

- **用法**：HLD hld(n)；add(u, v) 建无向树（点号 **1..n**）；work(root=1) 预处理；lca(u, v)、dist(u, v)、jump(u, k)（u 向上 k 步，不足返回 0）、isAncester(u, v)（u 是否为 v 祖先）、rootedParent(u, v)（以 u 为根时 v 的父亲）、rootedSize(u, v)（以 u 为根时 v 子树大小）、rootedLca(a, b, c)（三点 LCA，用异或消去）。
- **DFS 序**：in[u] 进栈时间戳、out[u] 出栈时间戳（子树对应区间 [in[u], out[u]]）；seq[t] 时间戳对应的点；重儿子排在 adj[u][0]，**重链在 in 上连续**，可配合线段树维护链信息。
- **复杂度**：预处理 O(n)，LCA / jump 单次 O(log n)。

```
struct HLD { // 重链剖分：1-indexed，根默认 1
    int n, cur;                                  // cur：DFS 序时间戳（从 1 开始）
    vector<int> sz, top, dep, fa, in, out, seq; // 子树大小 / 链头 / 深度 / 父亲 / 进栈 / 出栈 / 时间戳对应点
    vector<vector<int>> adj;                     // 邻接表（重儿子会被换到 adj[u][0]）

    HLD(int n) : n(n), cur(1), sz(n + 1), top(n + 1), dep(n + 1), fa(n + 1), in(n + 1), out(n + 1), seq(n + 1), adj(n + 1) {}

    void add(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void work(int root = 1) { // 预处理（默认根为 1）
        top[root] = root;
        dep[root] = 0;
        fa[root] = 0;
        dfs1(root);
        dfs2(root);
    }

    // 第一遍：求 fa、dep、sz，并把重儿子换到 adj[u][0]
    void dfs1(int u) {
        if (fa[u] != 0) {
            adj[u].erase(find(adj[u].begin(), adj[u].end(), fa[u])); // 删掉父边
        }
        sz[u] = 1;
        for (auto &v : adj[u]) {
            fa[v] = u;
            dep[v] = dep[u] + 1;
            dfs1(v);
            sz[u] += sz[v];
            if (sz[v] > sz[adj[u][0]]) { // 重儿子放最前
                swap(v, adj[u][0]);
            }
        }
    }

    // 第二遍：求 DFS 序 in/out/seq 与 top（重链头）
    void dfs2(int u) {
        in[u] = cur++;
        seq[in[u]] = u;
        for (auto v : adj[u]) {
            top[v] = v == adj[u][0] ? top[u] : v; // 重儿子继承链头
            dfs2(v);
        }
        out[u] = cur;
    }

    int lca(int u, int v) {
        while (top[u] != top[v]) { // 不同链：链头深的先跳
```

```
            if (dep[top[u]] < dep[top[v]])
                swap(u, v);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) // 同链：浅的为 LCA
            swap(u, v);
        return u;
    }

    int dist(int u, int v) { // 树上距离
        return dep[u] + dep[v] - 2 * dep[lca(u, v)];
    }

    int jump(int u, int k) { // u 向上跳 k 步；不足时返回 0
        if (dep[u] < k) {
            return 0;
        }

        int d = dep[u] - k; // 目标深度

        while (dep[top[u]] > d) { // 整链上跳
            u = fa[top[u]];
        }

        return seq[in[u] - dep[u] + d]; // 链内按 DFS 序直达
    }

    bool isAncester(int u, int v) { // u 是否为 v 的祖先（含自身）
        return in[u] ≤ in[v] && in[v] < out[u];
    }

    int rootedParent(int u, int v) { // 以 u 为根时 v 的父亲
        swap(u, v);
        if (u == v) {
            return u;
        }
        if (!isAncester(u, v)) {
            return fa[u];
        }
        // v 是根：找到 u 子树中 DFS 序最小的儿子方向
        auto it = upper_bound(adj[u].begin(), adj[u].end(), v, [&](int x, int y) {
            return in[x] < in[y];
        }) - 1;
        return *it;
    }

    int rootedSize(int u, int v) { // 以 u 为根时 v 的子树大小
        if (u == v) {
            return n;
        }
        if (!isAncester(v, u)) {
            return sz[v];
        }
        return n - sz[rootedParent(u, v)];
    }

    int rootedLca(int a, int b, int c) { // 三点 LCA：lca 两两异或
        return lca(a, b) ^ lca(b, c) ^ lca(c, a);
    }
};
```

数学 Math/

> 依赖 Template/ihxnan.hpp 的类型别名（ll、vi、lll = __int128 等）。 **冲突提醒**：MillerRabin.hpp 与 PollarRho.hpp 定义了相同符号（全局 Bases、mul、qmi、is_prime），**不能同时粘贴**；PollarRho 是 MillerRabin 的超集（多出 factorize），需要分解质因数时只用 PollarRho。 **全局状态**：EulerPrime 声明全局 vector<int> prime 与 vector<bool> is_prime。

组合数（阶乘 + 逆元，按需扩表）Comb.hpp

- **用法**：Comb comb(998244353); 自行实例化（传入模数）；comb.C(n, m) 组合数、comb.P(n, m) 排列数、comb.fac(x) / comb.ifac(x) 阶乘与阶乘逆元、comb.inv(x) 单点逆元。
- **复杂度**：init 摊还 O(n)，单次查询 O(1)（inv 单次 O(log mod)）。
- **注意**：C / P 越界参数（x < 0、y < 0、x < y）返回 0；fac / ifac 首次用到才扩表；模数须为质数（费马小定理求逆元）。

```
struct Comb // 组合数（阶乘 + 逆元，按需扩表）
{
    int n;                      // 当前已预处理到的上界
    ll mod;                     // 模数（须为质数）
    vector<ll> _fac, _inv;      // 阶乘表 _fac[i]=i!，逆元表 _inv[i]=(i!)^{-1}

    Comb(ll mod) : n(0), mod(mod), _fac{1}, _inv{1}
    {
    }

    ll qmi(ll a, ll b) // 快速幂
    {
        ll ans = 1;
        for (; b; b >>= 1, a = a * a % mod)
            if (b & 1)
                ans = ans * a % mod;
        return ans;
    }

    void init(int m) // 扩表到 m（增量计算）
    {
        if (m <= n)
            return;
        _fac.resize(m + 1);
        _inv.resize(m + 1);
        for (int i = n + 1; i <= m; ++i)
            _fac[i] = _fac[i - 1] * i % mod;
        _inv[m] = qmi(_fac[m], mod - 2);              // 阶乘逆元
        for (int i = m; i > n; --i)                   // 倒推各阶乘逆元
            _inv[i - 1] = _inv[i] * i % mod;
        n = m;
    }

    ll fac(int x) // x!（自动扩表）
    {
        if (x > n)
            init(x);
        return _fac[x];
    }

    ll inv(ll x) // 单点逆元（费马小定理）
    {
        return qmi(x, mod - 2);
    }

    ll ifac(int x) // (x!)^{-1}
    {
        if (x > n)
            init(x);
        return _inv[x];
    }

    ll C(int x, int y) // 组合数 C(x, y)，非法输入返回 0
    {
```

```
        if (x < 0 || y < 0 || x < y)
            return 0;
        return fac(x) * ifac(y) % mod * ifac(x - y) % mod;
    }

    ll P(int x, int y) // 排列数 P(x, y)
    {
        if (x < 0 || y < 0 || x < y)
            return 0;
        return fac(x) * ifac(x - y) % mod;
    }
};
```

欧拉线性筛（素数表） EulerPrime.hpp

- **用法**：`euler_prime(N);` 之后用全局 `prime`（升序素数表）与 `is_prime[0..N]`（素性）。
- **复杂度**：`O(n)`。
- **注意**：`prime` / `is_prime` 为**全局变量**，多次调用会追加/扩容而非重置；最小质因子性质：每个合数只被其最小质因子筛掉。

```
vector<int> prime;     // 升序素数表
vector<bool> is_prime; // is_prime[i]: i 是否为素数

void euler_prime(int n)
{
    is_prime.resize(n + 1, true);
    for (int i = 2; i ⩽ n; ++i)
    {
        if (is_prime[i])
            prime.push_back(i);
        for (auto &p : prime)
        {
            if (p * i > n)
                break;
            is_prime[i * p] = false;   // 用最小质因子 p 筛掉 i*p
            if (i % p == 0)            // p 是 i 的最小质因子，停止
                break;
        }
    }
}
```

扩展欧几里得（解不定方程） Exgcd.hpp

- **用法**：**代码片段**，整体粘贴进 solve() 内（clac 是捕获 [&] 的 lambda）。clac(a, b, c) 解 $a \cdot x + b \cdot y = c$。
- **输出**：有**正整数解**时输出 5 行——解的个数、x 最小值、y 最小值、x 最大值、y 最大值；**无正整数解**时输出 2 行——x 最小正整数值、y 最小正整数值；c % gcd != 0 输出 -1。
- **注意**：exgcd 返回 gcd(a, b) 并通过引用写出 x, y；ceil 为向上取整（处理负数的正确版本）。

```
template <class T> T sign(const T &a) // 符号函数
{
    return a == 0 ? 0 : (a < 0 ? -1 : 1);
}

template <class T> T ceil(const T &a, const T &b) // 向上取整（负数安全）
{
    T A = abs(a), B = abs(b);
    assert(b != 0);
    return sign(a) * sign(b) > 0 ? (A + B - 1) / B : -A / B;
}

int exgcd(int a, int b, int &x, int &y) // 求 gcd(a,b) 与一组解 x, y
{
    if (!b)
    {
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a % b, y, x); // 递归时交换 x/y
    y -= a / b * x;
    return d;
}

auto clac = [&](int a, int b, int c) -> void { // 解 a*x + b*y = c
    int x, y, d = exgcd(a, b, x, y);
    if (c % d != 0)
    {
        cout << -1 << endl; // 无整数解
        return;
    }
    x *= c / d, y *= c / d; // 缩放特解
    int p = b / d, q = a / d, k; // 通解：x + kp, y - kq
    if (x < 0)
    {
        k = ceil(1 - x, p);
        x += p * k;
        y -= q * k;
    }
    else if (x >= 0)
    { // 将x提高到最小正整数
        k = (x - 1) / p;
        x -= p * k; // 将x降低到最小正整数
        y += q * k;
    }
    if (y > 0)
    {                                       // 有正整数解
        cout << (y - 1) / q + 1 << endl;    // 将y减到1的方案数即为解的个数
        cout << x << endl;                  // 当前的x即为最小正整数x
        cout << (y - 1) % q + 1 << endl;    // 将y取到最小正整数
        cout << x + (y - 1) / q * p << endl; // 将x提升到最大
        cout << y << endl;                  // 特解即为y最大值
    }
    else
    {                                       // 无整数解
```

```
        cout << x << endl;                        // 当前的x即为最小的正整数x
        cout << y + q * ceil(1 - y, q) << endl; // 将y提高到正整数
    }
};
```

Miller-Rabin 素性测试 MillerRabin.hpp

- **用法**：is_prime(n)，n 为 long long。
- **复杂度**：单次 O(log^2 n)。
- **注意**：基准 {2,3,5,7,11,13,17,19,23} 对 n < 3.3×10^24 判定正确（覆盖全部 ll）；与 PollarRho.hpp **符号冲突**，同文件二选一。

```
vector<int> Bases{2, 3, 5, 7, 11, 13, 17, 19, 23};

ll mul(ll a, ll b, ll m) // 模乘（__int128 防溢出）
{
    return lll(a) * b % m;
}

ll qmi(ll a, ll b, ll mod) // 快速幂取模
{
    ll res = 1;
    for (; b; b >>= 1, a = mul(a, a, mod))
        if (b & 1)
            res = mul(res, a, mod);
    return res;
}

bool is_prime(ll n)
{
    if (n < 2)
        return false;
    int s = __builtin_ctzll(n - 1); // n-1 = d * 2^s
    ll d = (n - 1) >> s;
    for (auto &p : Bases)
    {
        if (p == n)
            return true;
        ll x = qmi(p, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i) // 二次探测链
        {
            x = mul(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false; // 该基准判定为合数
    }
    return true;
}
```

Pollard-Rho 质因数分解（含 Miller-Rabin） `PollarRho.hpp`

- **用法**：`factorize(n)` 返回升序**带重复**的质因子列表（如 `12 → {2, 2, 3}`）；`is_prime(n)` 可单独使用。
- **复杂度**：期望 `O(n^{1/4})`（每次找一个因子）。
- **注意**：是 `MillerRabin.hpp` 的超集，符号冲突二选一；`n <= 10000` 时直接试除；Floyd 判环 + 乘积批量取 `gcd` 优化。

```
vector<int> Bases{2, 3, 5, 7, 11, 13, 17, 19, 23};

ll mul(ll a, ll b, ll m) // 模乘（__int128 防溢出）
{
    return lll(a) * b % m;
}

ll qmi(ll a, ll b, ll mod) // 快速幂取模
{
    ll res = 1;
    for (; b; b >>= 1, a = mul(a, a, mod))
        if (b & 1)
            res = mul(res, a, mod);
    return res;
}

bool is_prime(ll n) // Miller-Rabin 素性测试
{
    if (n < 2)
        return false;
    int s = __builtin_ctzll(n - 1); // n-1 = d * 2^s
    ll d = (n - 1) >> s;
    for (auto &p : Bases)
    {
        if (p == n)
            return true;
        ll x = qmi(p, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool ok = false;
        for (int i = 0; i < s - 1; ++i)
        {
            x = mul(x, x, n);
            if (x == n - 1)
            {
                ok = true;
                break;
            }
        }
        if (!ok)
            return false;
    }
    return true;
}

vector<ll> factorize(ll n) // 返回升序带重复质因子
{
    vector<ll> factors;
    auto dfs = [&](auto &&self, ll n) {
        if (n <= 10000) // 小数直接试除
        {
            for (int i = 2; i * i <= n; ++i)
                for (; n % i == 0; n /= i)
                    factors.push_back(i);
            if (n > 1)
                factors.push_back(n);
```

```
            return;
        }
        if (is_prime(n))
        {
            factors.push_back(n);
            return;
        }
        ll c = 1; // 随机迭代函数 (x^2 + c) mod n
        auto rho_func = [&](ll x) { return (mul(x, x, n) + c) % n; };
        ll x0 = 2;
        while (true)
        {
            ll x = x0;
            ll y = x0;
            ll d = 1;
            ll power = 1, lam = 0; // Brent 优化：2^k 步一检查
            ll v = 1;              // 途中乘积
            while (d == 1)
            {
                y = rho_func(y);
                ++lam;
                v = mul(v, abs(x - y), n); // 批量累积差值
                if (lam % 127 == 0)        // 每 127 步取一次 gcd
                {
                    d = gcd(v, n);
                    v = 1;
                }
                if (power == lam) // Floyd 倍增检查点
                {
                    x = y;
                    power *= 2;
                    lam = 0;
                    d = gcd(v, n);
                    v = 1;
                }
            }
            if (d != n) // 找到真因子，分治
            {
                self(self, d);
                self(self, n / d);
                return;
            }
            ++x0; // 该 (x0, c) 失败，换参数重试
            ++c;
        }
    };
    dfs(dfs, n);
    sort(factors.begin(), factors.end());
    return factors;
}
```

线性基 LinearBasis.hpp

- **用法**：LinearBasis<unsigned long long> lb;（建议**无符号整型**）；insert(x) 插入（返回是否新增基）；find(x) 判断 x 能否被表出；max() / min() 最大 / 最低位基；kth(k) 第 k 小可表出值；rebuild() 化为最简标准形。
- **复杂度**：单次 O(BITS)。
- **注意**：kth 从 1 开始计数；插过可被表出的元素（insert 返回 false）后 zero = true（空集 0 可表出），kth 会自动跳过空集；k 超出范围返回 T(-1)。

```
template <typename T> struct LinearBasis
{
    const int BITS = sizeof(T) * 8; // 位宽（如 ull 为 64）
    int sz;       // 基的个数
    bool zero;    // 0 是否可被表出（存在冗余插入）
    vector<T> basis;    // basis[i] 最高位为 i
    vector<T> standard; // 化简后的标准形（升序）

    LinearBasis() : sz(0), zero(false), basis(BITS, T(0))
    {
    }

    // 高斯消元化标准形：basis[i] 只含自己特有的低位
    void rebuild()
    {
        for (int i = BITS - 1; i >= 0; i--)
            for (int j = i - 1; j >= 0; j--)
                if (basis[i] >> j & 1)
                    basis[i] ^= basis[j];

        standard.clear();
        for (int i = 0; i < BITS; i++)
            if (basis[i])
                standard.push_back(basis[i]);
    }

    void clear() // 清空
    {
        sz = 0;
        zero = false;
        basis.assign(BITS, T(0));
        standard.clear();
    }

    bool insert(T x) // 插入 x；返回 false 表示 x 可被现有基表出
    {
        for (int i = BITS - 1; i >= 0; i--)
            if (x >> i & 1)
            {
                if (!basis[i])
                {
                    basis[i] = x, ++sz; // 该位空缺，直接成为基
                    return true;
                }
                x ^= basis[i]; // 消去最高位继续
            }
        zero = true; // x 被完全消成 0
        return false;
    }

    bool find(T x) // x 能否被表出
    {
        for (int i = BITS - 1; i >= 0; i--)
            if (x >> i & 1)
```

```
            {
                if (!basis[i])
                    return false;
                x ^= basis[i];
            }
        return true;
    }

    T max() // 最大可表出值: 贪心
    {
        T res = 0;
        for (int i = BITS - 1; i >= 0; --i)
            res = std::max(res, res ^ basis[i]);
        return res;
    }

    T min() // 最低位的非零基
    {
        for (int i = 0; i < BITS; ++i)
            if (basis[i])
                return basis[i];
        return 0;
    }

    int size() // 基的个数
    {
        return sz;
    }

    T kth(T k) // 第 k 小可表出值 (1-based) ; 越界返回 T(-1)
    {
        if (standard.empty())
            rebuild();
        if (zero)
            --k;      // 跳过空集
        T res = T(0);
        for (auto &x : standard)
        {
            if (k & 1) // k 的第 i 位决定是否取第 i 个基
                res ^= x;
            k >>= 1;
        }
        return k ? T(-1) : res; // k 未用尽: 越界
    }
};
```

计算几何（点 / 线 / 多边形 / 半平面交） Point.hpp

- **用法**：Point<T> 为模板，需自行指定坐标类型并起别名（如 using P = Point<long double>;）后使用；整型坐标（ll）可精确运算（叉积判向），浮点注意精度——本文件不设 eps，所有比较均为精确 ==，故一般优先用整型。
- **内容**：Point（向量运算、流 IO）、Line（两点式）、dot 点积、cross 叉积、length 模长、normalize 单位化、parallel 平行、distancePL 点到直线、distancePS 点到线段、rotate 逆时针旋转 90°、sgn 极角半区、pointOnLineLeft 点在直线左侧、lineIntersection 直线交点、pointOnSegment 点在线段上、pointInPolygon 点在多边形内（含边界）、segmentIntersection 线段相交（返回 {类型, p1, p2}：0 不相交 / 1 严格相交 / 2 重叠 / 3 端点相交）、distanceSS 线段距离、segmentInPolygon 线段在多边形内、hp 半平面交（返回交点列表，逆时针？按排序方向给出多边形顶点）。

```
template <class T>
struct Point { // 二维点 / 向量
    T x;
    T y;
    Point(const T &x_ = 0, const T &y_ = 0) : x(x_), y(y_) {}

    template <class U>
    operator Point<U>() { // 类型转换
        return Point<U>(U(x), U(y));
    }
    // ---- 复合赋值（& 限定：只作用于具名对象，不作用于临时量）----
    Point &operator+=(const Point &p) & { // 向量加
        x += p.x;
        y += p.y;
        return *this;
    }
    Point &operator-=(const Point &p) & { // 向量减
        x -= p.x;
        y -= p.y;
        return *this;
    }
    Point &operator*=(const T &v) & { // 数乘
        x *= v;
        y *= v;
        return *this;
    }
    Point &operator/=(const T &v) & { // 数除
        x /= v;
        y /= v;
        return *this;
    }
    // ---- 二元运算 / 取负（值语义，返回新对象）----
    Point operator-() const { // 取负向量
        return Point(-x, -y);
    }
    friend Point operator+(Point a, const Point &b) { // a + b
        return a += b;
    }
    friend Point operator-(Point a, const Point &b) { // a - b
        return a -= b;
    }
    friend Point operator*(Point a, const T &b) { // 点 * 数
        return a *= b;
    }
    friend Point operator/(Point a, const T &b) { // 点 / 数
        return a /= b;
    }
    friend Point operator*(const T &a, Point b) { // 数 * 点
        return b *= a;
    }
    // ---- 比较与流 IO ----
```

```
    friend bool operator==(const Point &a, const Point &b) { // 逐分量相等
        return a.x == b.x && a.y == b.y;
    }
    friend std::istream &operator>>(std::istream &is, Point &p) { // 读入
        return is >> p.x >> p.y;
    }
    friend std::ostream &operator<<(std::ostream &os, const Point &p) { // 输出 (x, y)
        return os << "(" << p.x << ", " << p.y << ")";
    }
};

template <class T>
struct Line { // 两点式直线 / 线段
    Point<T> a;
    Point<T> b;
    Line(const Point<T> &a_ = Point<T>(), const Point<T> &b_ = Point<T>()) : a(a_), b(b_) {}
};

template <class T>
T dot(const Point<T> &a, const Point<T> &b) { // 点积
    return a.x * b.x + a.y * b.y;
}

template <class T>
T cross(const Point<T> &a, const Point<T> &b) { // 叉积
    return a.x * b.y - a.y * b.x;
}

template <class T>
T square(const Point<T> &p) { // 模长平方
    return dot(p, p);
}

template <class T>
double length(const Point<T> &p) { // 模长
    return std::sqrt(square(p));
}

template <class T>
double length(const Line<T> &l) { // 线段长
    return length(l.a - l.b);
}

template <class T>
Point<T> normalize(const Point<T> &p) { // 单位向量
    return p / length(p);
}

template <class T>
bool parallel(const Line<T> &l1, const Line<T> &l2) { // 平行（含共线）
    return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}

template <class T>
double distance(const Point<T> &a, const Point<T> &b) { // 两点距离
    return length(a - b);
}

template <class T>
double distancePL(const Point<T> &p, const Line<T> &l) { // 点到直线距离
    return std::abs(cross(l.a - l.b, l.a - p)) / length(l);
}

template <class T>
```

```
double distancePS(const Point<T> &p, const Line<T> &l) { // 点到线段距离
    if (dot(p - l.a, l.b - l.a) < 0) { // 在 a 外侧投影
        return distance(p, l.a);
    }
    if (dot(p - l.b, l.a - l.b) < 0) { // 在 b 外侧投影
        return distance(p, l.b);
    }
    return distancePL(p, l); // 垂足落在线段上
}

template <class T>
Point<T> rotate(const Point<T> &a) { // 逆时针旋转 90°
    return Point(-a.y, a.x);
}

template <class T>
int sgn(const Point<T> &a) { // 极角所在半区（上半 / x 正半轴为 1）
    return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}

template <class T>
bool pointOnLineLeft(const Point<T> &p, const Line<T> &l) { // p 在直线 l 左侧
    return cross(l.b - l.a, p - l.a) > 0;
}

template <class T>
Point<T> lineIntersection(const Line<T> &l1, const Line<T> &l2) { // 直线交点
    return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b));
}

template <class T>
bool pointOnSegment(const Point<T> &p, const Line<T> &l) { // p 在线段 l 上
    return cross(p - l.a, l.b - l.a) == 0 && std::min(l.a.x, l.b.x) <= p.x && p.x <= std::max(l.a.x, l.
b.x)
        && std::min(l.a.y, l.b.y) <= p.y && p.y <= std::max(l.a.y, l.b.y);
}

template <class T>
bool pointInPolygon(const Point<T> &a, const std::vector<Point<T>> &p) { // 点在多边形内（含边界）
    int n = p.size();
    for (int i = 0; i < n; i++) { // 在边上直接算在内
        if (pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
            return true;
        }
    }

    int t = 0;                  // 射线（水平向右）穿越计数
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
            t ^= 1;
        }
        if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
            t ^= 1;
        }
    }

    return t == 1; // 奇数次穿越：内部
}

// 线段相交判定，返回 {类型, p1, p2}:
// 0 : 不相交
// 1 : 严格相交（交于内部一点）
```

```
// 2 : 重叠 (p1, p2 为重叠段端点)
// 3 : 相交于端点 (或交点为端点)
template <class T>
std::tuple<int, Point<T>, Point<T>> segmentIntersection(const Line<T> &l1, const Line<T> &l2) {
    // 快速排斥: 包围盒不相交
    if (std::max(l1.a.x, l1.b.x) < std::min(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::min(l1.a.x, l1.b.x) > std::max(l2.a.x, l2.b.x)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::max(l1.a.y, l1.b.y) < std::min(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (std::min(l1.a.y, l1.b.y) > std::max(l2.a.y, l2.b.y)) {
        return {0, Point<T>(), Point<T>()};
    }
    if (cross(l1.b - l1.a, l2.b - l2.a) == 0) { // 平行
        if (cross(l1.b - l1.a, l2.a - l1.a) != 0) { // 不共线
            return {0, Point<T>(), Point<T>()};
        } else { // 共线: 求重叠区间
            auto maxx1 = std::max(l1.a.x, l1.b.x);
            auto minx1 = std::min(l1.a.x, l1.b.x);
            auto maxy1 = std::max(l1.a.y, l1.b.y);
            auto miny1 = std::min(l1.a.y, l1.b.y);
            auto maxx2 = std::max(l2.a.x, l2.b.x);
            auto minx2 = std::min(l2.a.x, l2.b.x);
            auto maxy2 = std::max(l2.a.y, l2.b.y);
            auto miny2 = std::min(l2.a.y, l2.b.y);
            Point<T> p1(std::max(minx1, minx2), std::max(miny1, miny2));
            Point<T> p2(std::min(maxx1, maxx2), std::min(maxy1, maxy2));
            if (!pointOnSegment(p1, l1)) {
                std::swap(p1.y, p2.y);
            }
            if (p1 == p2) {
                return {3, p1, p2}; // 交于一点
            } else {
                return {2, p1, p2}; // 重叠
            }
        }
    }
    // 跨立实验: 互相在两侧才相交
    auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
    auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
    auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
    auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);

    if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) || (cp3 < 0 && cp4 < 0)) {
        return {0, Point<T>(), Point<T>()};
    }

    Point p = lineIntersection(l1, l2);
    if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) {
        return {1, p, p}; // 严格相交
    } else {
        return {3, p, p}; // 交在端点
    }
}

template <class T>
double distanceSS(const Line<T> &l1, const Line<T> &l2) { // 线段最近距离
    if (std::get<0>(segmentIntersection(l1, l2)) != 0) {
        return 0.0; // 相交则为 0
    }
```

```
    // 不相交：四个端点到对方线段距离的最小值
    return std::min({distancePS(l1.a, l2), distancePS(l1.b, l2), distancePS(l2.a, l1), distancePS(l2.b,
l1)});
}

template <class T>
bool segmentInPolygon(const Line<T> &l, const std::vector<Point<T>> &p) { // 线段是否完全在多边形内
    int n = p.size();
    if (!pointInPolygon(l.a, p)) {
        return false;
    }
    if (!pointInPolygon(l.b, p)) {
        return false;
    }
    for (int i = 0; i < n; i++) { // 逐边检查与线段的关系
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto w = p[(i + 2) % n];
        auto [t, p1, p2] = segmentIntersection(l, Line(u, v));

        if (t == 1) { // 从内部穿出：不行
            return false;
        }
        if (t == 0) {
            continue;
        }
        if (t == 2) { // 与边重叠：沿多边形方向拐弯则不行
            if (pointOnSegment(v, l) && v != l.a && v != l.b) {
                if (cross(v - u, w - v) > 0) {
                    return false;
                }
            }
        } else { // 交于点：分情况检查是否贴边
            if (p1 != u && p1 != v) { // 交点在边内部：穿出
                if (pointOnLineLeft(l.a, Line(v, u))
                    || pointOnLineLeft(l.b, Line(v, u))) {
                    return false;
                }
            } else if (p1 == v) { // 交点是边的右端点
                if (l.a == v) {
                    if (pointOnLineLeft(u, l)) {
                        if (pointOnLineLeft(w, l)
                            && pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, l)
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                } else if (l.b == v) {
                    if (pointOnLineLeft(u, Line(l.b, l.a))) {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            && pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                } else {
```

```
                    if (pointOnLineLeft(u, l)) {
                        if (pointOnLineLeft(w, Line(l.b, l.a))
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    } else {
                        if (pointOnLineLeft(w, l)
                            || pointOnLineLeft(w, Line(u, v))) {
                            return false;
                        }
                    }
                }
            }
        }
    }
    return true;
}

template <class T>
std::vector<Point<T>> hp(std::vector<Line<T>> lines) { // 半平面交，返回交多边形顶点
    // 按极角排序（同向按叉积，限制在左侧）
    std::sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
        auto d1 = l1.b - l1.a;
        auto d2 = l2.b - l2.a;

        if (sgn(d1) != sgn(d2)) { // 不同半区：上半区在前
            return sgn(d1) == 1;
        }

        return cross(d1, d2) > 0;
    });

    std::deque<Line<T>> ls; // 当前有效半平面
    std::deque<Point<T>> ps; // 相邻半平面交点
    for (auto l : lines) {
        if (ls.empty()) {
            ls.push_back(l);
            continue;
        }

        // 交点已不在新半平面内：弹出队尾
        while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
            ps.pop_back();
            ls.pop_back();
        }

        // 弹出队首
        while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
            ps.pop_front();
            ls.pop_front();
        }

        if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) { // 与队尾平行
            if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) { // 同向：保留更紧的
                if (!pointOnLineLeft(ls.back().a, l)) {
                    assert(ls.size() == 1);
                    ls[0] = l;
                }
                continue;
            }
            return {}; // 反向平行：无交
        }

        ps.push_back(lineIntersection(ls.back(), l));
```

```
        ls.push_back(l);
    }

    // 收尾：队首半平面检查队尾交点
    while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
        ps.pop_back();
        ls.pop_back();
    }
    if (ls.size() ⩽ 2) { // 不足三条：无有效交
        return {};
    }
    ps.push_back(lineIntersection(ls[0], ls.back())); // 闭合多边形

    return std::vector(ps.begin(), ps.end());
}
```

字符串 String/

> 依赖 `Template/ihxnan.hpp` 的类型别名（`ll`、`ul`、`vi`、`vvi` 等）。 **索引约定不一**：`KMP.hpp` 三个函数均为 **1-indexed**（`str[0]` 为占位符）；其余（`StrHash`、`SqAM`、`SuffixArray`）为 **0-indexed**；`Trie` / `AhoCorasick` 为节点编号约定，见各自说明。

KMP / Z 函数（1-indexed） KMP.hpp

- **用法**：传入的 `str` 需为 1-indexed（`str[0]` 占位，实际字符在 `1..n`，如 `"$[文本]"` 风格构造）。
  - `get_fail(str)`：返回 `fail[1..n]`，`fail[i]` 为 `str[1..i]` 的最长相等真前后缀长度（`fail[0]` 为 `0`）。
  - `get_z(str)`：Z 函数，`z[1] = n`，`z[i]` 为 `str` 与 `str[i..]` 的 LCP 长度。
  - `get_tr(str, fail)`：KMP 自动机转移表，`tr[i][c]` 为位置 `i` 后接字符 `c` 时的转移目标。
- **复杂度**：均为 `O(n)`（`get_tr` 乘上字符集 26）。

```
vector<int> get_fail(const string &str) // KMP 失配数组
{
    int n = str.size() - 1; // str[0] 为占位符
    vector<int> fail(n + 1);
    for (int i = 2, j = 0; i ⩽ n; ++i)
    {
        while (j && str[i] ≠ str[j + 1]) // 失配沿 fail 回跳
            j = fail[j];
        if (str[i] == str[j + 1])
            ++j;
        fail[i] = j;
    }
    return fail;
}

vector<vector<int>> get_tr(const string &str, const vector<int> &fail) // KMP 自动机
{
    int n = fail.size() - 1;
    vector<vector<int>> tr(fail.size(), vector<int>(26));
    for (int i = 0; i < n; ++i)
    {
        for (int j = 0; j < 26; ++j)
            tr[i][j] = tr[fail[i]][j];  // 继承 fail 的转移
        tr[i][str[i + 1] - 'a'] = i;    // 匹配字符的转移
    }
    return tr;
}

vector<int> get_z(const string &str) // Z 函数
{
    int n = str.size() - 1; // str[0] 为占位符
    vector<int> z(n + 1);
    if (n == 0)
        return z;
    z[1] = n;
    for (int i = 2, l = 0, r = 0; i ⩽ n; i++)
    {
        if (i ⩽ r)
            z[i] = min(r - i + 1, z[i - l + 1]); // 利用窗口 [l, r] 信息
        while (i + z[i] ⩽ n && str[1 + z[i]] == str[i + z[i]])
            z[i]++; // 暴力扩展
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1; // 更新右端最远窗口
    }
    return z;
}
```

字符串哈希（自然溢出，0-indexed）StrHash.hpp

- **用法**：StrHash hs(s);（下标 0..n-1，区间均为闭区间 [l, r]）；get(l, r) 正向哈希、rget(l, r) 反向哈希、is_palindrome(l, r) 回文判断、same(l1, r1, l2, r2) 子串相等、link(l1, r1, l2, r2) 拼接两段哈希、full() 整串哈希。
- **实现**：底数 13331，**自然溢出**（ul = unsigned long long，模 2^64）。
- **注意**：自然溢出有被构造数据卡的风险，赛时可把 base 换成随机数。

```
struct StrHash { // 字符串哈希（自然溢出，0-indexed，区间闭 [l, r]）
    ul base;               // 底数
    vector<ul> pow, h, rh; // pow: 幂次；h: 正向前缀哈希；rh: 反向后缀哈希

    StrHash(const string &str) : base(13331)
    {
        int n = str.size();
        pow.resize(n + 1);
        h.resize(n + 1);
        rh.resize(n + 1);
        pow[0] = 1, h[0] = 0, rh[n] = 0;
        for (int i = 0; i < n; i++) // 正向：h[i+1] = s[0..i] 的哈希
        {
            pow[i + 1] = pow[i] * base;
            h[i + 1] = h[i] * base + str[i];
        }
        for (int i = n - 1; i >= 0; i--) // 反向：rh[i] = s[i..n-1] 的哈希
            rh[i] = rh[i + 1] * base + str[i];
    }

    ul full() // 整串哈希
    {
        return h.back();
    }

    ul get(int l, int r) // 子串 s[l..r] 正向哈希
    {
        return h[r + 1] - h[l] * pow[r - l + 1];
    }

    ul rget(int l, int r) // 子串 s[l..r] 反向哈希
    {
        return rh[l] - rh[r + 1] * pow[r - l + 1];
    }

    ul link(int l1, int r1, int l2, int r2) // 拼接 s[l1..r1] + s[l2..r2]
    {
        return get(l1, r1) * pow[r2 - l2 + 1] + get(l2, r2);
    }

    bool same(int l1, int r1, int l2, int r2) // 两子串是否相等
    {
        return get(l1, r1) == get(l2, r2);
    }

    bool is_palindrome(int l, int r) // s[l..r] 是否回文
    {
        return get(l, r) == rget(l, r);
    }
};
```

Trie（62 字符集，支持计数 / 删除） Trie.hpp

- **用法**：Trie tr(预计节点数);；insert(s) 插入（pass 沿途 +1，end 终点 +1）；search(s) 完整单词 s 的个数；query(s) 以 s 为前缀的单词个数；del(s) 删除一个 s（沿 pass 减、无经过即剪枝）；clear() 清空复用。
- **字符集**：a-z（0-25）、A-Z（26-51）、0-9（52-61），共 62。
- **注意**：构造参数为节点数上限（内部开 n+5 个节点），传"字符串总长"即可；节点 0 为根。

```
struct Trie // 字典树（62 字符集：a-z A-Z 0-9，节点 0 为根）
{
    int tot;                        // 当前节点计数
    vector<int> pass, end;          // pass: 经过该节点的单词数；end: 以该节点结尾的单词数
    vector<vector<int>> trie;       // trie[u][c]: u 的字符 c 儿子（0 表示无）

    int get(const char &x) // 字符映射到 0..61
    {
        if (x ⩾ 'a' && x ⩽ 'z')
            return x - 'a';
        if (x ⩾ 'A' && x ⩽ 'Z')
            return x - 'A' + 26;
        return x - '0' + 52;
    }

    Trie(int n) : tot(0), pass(n + 5), end(n + 5), trie(n + 5, vector<int>(62))
    {
    }

    void insert(const string &str)
    {
        int cur = 0;
        ++pass[cur]; // 根也计数
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                trie[cur][x] = ++tot; // 无儿子则新建
            cur = trie[cur][x];
            ++pass[cur];
        }
        ++end[cur];
    }

    int search(const string &str) // 完整单词 str 的个数
    {
        int cur = 0;
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                return 0;
            cur = trie[cur][x];
        }
        return end[cur];
    }

    int query(const string &str) // 以 str 为前缀的单词个数
    {
        int cur = 0;
        for (auto &p : str)
        {
            int x = get(p);
            if (!trie[cur][x])
                return 0;
```

```
            cur = trie[cur][x];
        }
        return pass[cur];
    }

    void del(const string &str) // 删除一个 str（需已插入）
    {
        if (search(str) > 0)
        {
            int cur = 0;
            --pass[cur];
            for (auto &p : str)
            {
                int x = get(p);
                if (!--pass[trie[cur][x]]) // 无经过：直接剪枝
                {
                    trie[cur][x] = 0;
                    return;
                }
                cur = trie[cur][x];
            }
            --end[cur];
        }
    }

    void clear() // 清空所有节点（可复用）
    {
        for (int i = 0; i <= tot; ++i)
        {
            pass[i] = end[i] = 0;
            fill(trie[i].begin(), trie[i].end(), 0);
        }
        tot = 0;
    }
};
```

AC 自动机（小写字母）AhoCorasick.hpp

- **用法**：AhoCorasick ac;；对每个模式串 add(s)（返回末节点，可在此打标记）；全部加完后 work() BFS 建 fail 指针并补全转移；之后 next(p, c) 在自动机上转，link(p) 取 fail。
- **结构**：节点 0 为哨兵（next 全指向根，len = -1），节点 1 为根；len 为节点深度（对应模式串前缀长度）。
- **常用套路**：文本沿 next 转移；fail 树上子树求和 / 拓扑序 DP 统计各模式串出现次数。
- **注意**：仅支持小写字母（ALPHABET = 26）。

```
struct AhoCorasick
{
    static constexpr int ALPHABET = 26;
    struct Node
    {
        int len;                        // 节点深度（该前缀长度）
        int link;                       // fail 指针
        array<int, ALPHABET> next;      // 自动机转移（已补全）
        Node() : len{0}, link{0}, next{}
        {
        }
    };

    vector<Node> tree;

    AhoCorasick()
    {
        init();
    }

    void init() // 节点 0 哨兵 + 节点 1 根
    {
        tree.assign(2, Node());
        tree[0].next.fill(1); // 哨兵所有转移指向根
        tree[0].len = -1;
    }

    int newNode()
    {
        tree.emplace_back();
        return tree.size() - 1;
    }

    int add(const string &str) // 插入模式串，返回末节点
    {
        int p = 1; // 从根出发
        for (auto c : str)
        {
            int x = c - 'a';
            if (tree[p].next[x] == 0)
            {
                tree[p].next[x] = newNode();
                tree[tree[p].next[x]].len = tree[p].len + 1;
            }
            p = tree[p].next[x];
        }
        return p;
    }

    void work() // BFS 建 fail 指针，并把缺省转移补全
    {
        queue<int> que;
        que.push(1);
```

```
        while (!que.empty())
        {
            int x = que.front();
            que.pop();

            for (int i = 0; i < ALPHABET; i++)
            {
                if (tree[x].next[i] == 0)
                {
                    tree[x].next[i] = tree[tree[x].link].next[i]; // 补全转移
                }
                else
                {
                    tree[tree[x].next[i]].link = tree[tree[x].link].next[i]; // fail
                    que.push(tree[x].next[i]);
                }
            }
        }
    }

    int next(int p, int x) // p 状态遇字符 x 的转移
    {
        return tree[p].next[x];
    }

    int link(int p) // fail 指针
    {
        return tree[p].link;
    }

    int len(int p) // 节点深度
    {
        return tree[p].len;
    }

    int size() // 节点总数
    {
        return tree.size();
    }
};
```

子序列自动机 SequenceAM.hpp

- **用法**：SqAM sam(s);（0-indexed）；find(t) 判断 t 是否为 s 的**子序列**。
- **实现**：next[i][c] = 位置 i 及之后第一个字符 c 出现的下标（无则 -1）；匹配时 idx = next[idx][c] + 1。
- **复杂度**：预处理 O(26n) 时间与空间，匹配 O(|t|)。
- **注意**：内存开销 26n 个 int，n 很大时注意空间（可用 n log 26 的倍增 / 逐字符贪心替代）。

```
struct SqAM // 子序列自动机：0-indexed
{
    int n;                      // 串长
    vector<vector<int>> next; // next[i][c]: 位置 i 及之后首个字符 c 的下标（无则 -1）
    SqAM() = default;
    SqAM(const string &str)
    {
        init(str);
    }
    void clear()
    {
        n = 0, next.clear();
    }
    void init(const string &str) // 预处理：从右往左递推
    {
        n = str.size();
        next.resize(n + 1, vector<int>(26, -1));
        for (int i = n - 1; i >= 0; --i)
        {
            for (int j = 0; j < 26; ++j)
                next[i][j] = next[i + 1][j];   // 继承右侧
            next[i][str[i] - 'a'] = i;          // 覆盖当前字符
        }
    }
    bool find(const string &str) // str 是否为 s 的子序列
    {
        int idx = 0;
        for (auto p : str)
        {
            idx = next[idx][p - 'a'] + 1; // 跳到下一个匹配位置之后
            if (idx <= 0)                 // next 为 -1 时加 1 得 0：失配
                return false;
        }
        return true;
    }
};
```

后缀数组（倍增 + 计数排序，0-indexed） SuffixArray.hpp

- **用法**：SuffixArray sa(s);（s 为 0-indexed）；成员 sa[i]：按字典序第 i 小的后缀起点；rk[i]：后缀 i 的排名；lc[i]：lcp(sa[i], sa[i+1])（长度 n-1）。
- **复杂度**：O(n log n)。
- **注意**：n == 1 时 lc 为空；常见配合——任意两后缀 LCP 为 lc 数组上 min(rk[u] .. rk[v]-1)（RMQ/ST 表）。

```
struct SuffixArray // 后缀数组（倍增 + 计数排序）：0-indexed
{
    int n;                  // 串长
    vector<int> sa, rk, lc; // sa: 名次→后缀起点；rk: 后缀起点→名次；lc[i]=lcp(sa[i], sa[i+1])，长度 n
-1

    SuffixArray(const string &s)
    {
        n = s.length();
        sa.resize(n);
        lc.resize(n - 1);
        rk.resize(n);
        iota(sa.begin(), sa.end(), 0);
        sort(sa.begin(), sa.end(), [&](int a, int b) { return s[a] < s[b]; }); // 按首字符排序
        rk[sa[0]] = 0;
        for (int i = 1; i < n; i++) // 首字符排名
            rk[sa[i]] = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
        int k = 1;
        vector<int> tmp;
        vector<int> cnt(n);
        tmp.reserve(n);
        while (rk[sa[n - 1]] < n - 1) // 排名未全部分开，倍增
        {
            // 以长度 k 的排名为关键字，对长度 2k 排序
            // 1) 生成第二关键字顺序：越界后缀最小，其余为 sa[i]-k
            tmp.clear();
            for (int i = 0; i < k; i++)
                tmp.push_back(n - k + i);
            for (auto i : sa)
                if (i >= k)
                    tmp.push_back(i - k);
            // 2) 对第一关键字计数排序
            fill(cnt.begin(), cnt.end(), 0);
            for (int i = 0; i < n; i++)
                cnt[rk[i]]++;
            for (int i = 1; i < n; i++)
                cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--)
                sa[--cnt[rk[tmp[i]]]] = tmp[i];
            // 3) 重算长度 2k 的排名
            swap(rk, tmp);
            rk[sa[0]] = 0;
            for (int i = 1; i < n; i++)
                rk[sa[i]] = rk[sa[i - 1]] +
                            (tmp[sa[i - 1]] < tmp[sa[i]] || sa[i - 1] + k == n || tmp[sa[i - 1] + k] < t
mp[sa[i] + k]);
            k *= 2;
        }
        // O(n) 求 height 数组 lc（利用 h[i] >= h[i-1]-1）
        for (int i = 0, j = 0; i < n; i++)
            if (rk[i] == 0)
                j = 0;
            else
            {
                for (j -= (j > 0); i + j < n && sa[rk[i] - 1] + j < n && s[i + j] == s[sa[rk[i] - 1] +
```

```
j]; j++)
                    ;
                lc[rk[i] - 1] = j;
            }
    }
};
```

对拍 对拍/

> **用法**：把 data.cpp （数据生成器）、ans.cpp （标准 / 暴力解）、test.cpp （待测程序）放到同一目录，双击或命令行运行 dp.bat ；可选参数为对拍轮数，默认 100 。
>
> - 全部通过：退出码 0 ；出现**编译 / 运行失败**或**输出不一致**：退出码 1 （可被上层脚本判断）。
> - 仅 Windows 的 cmd （ .bat ）可用；依赖 PATH 中的 g++ （MinGW）。
> - 计时： Measure-Command { dp.bat [n] | Out-Host } 。

Windows 对拍脚本 `dp.bat`

- **流程**：编译三个程序 → 循环 `N` 轮：生成数据 → 分别运行 `ans` / `test` → 逐字节比较输出 → 不一致则打印 input / expected / got 并退出。
- **参数**：`dp.bat [轮数]`，缺省 `N=100`。
- **比较**：`fc /b` 为二进制精确比较，**行尾空格 / 多余空行 / 末尾换行**差异都会判错。
- **清理**：结束时删除 `data.exe` / `ans.exe` / `test.exe`。

```
@echo off

rem Simple timer
rem Measure-Command { dp.bat [n] | Out-Host }

set N=100
if not "%~1"=="" set N=%~1

echo Compiling ...
g++ -O2 data.cpp -o data.exe  || goto :fail
g++ -O2 ans.cpp  -o ans.exe   || goto :fail
g++ -O2 test.cpp -o test.exe  || goto :fail
echo Compile OK
echo.

set /a cnt=0

:loop
set /a cnt+=1

data.exe > input.txt
if errorlevel 1 goto :fail

ans.exe  < input.txt > ans_out.txt
if errorlevel 1 goto :fail

test.exe < input.txt > test_out.txt
if errorlevel 1 goto :fail

fc /b ans_out.txt test_out.txt >nul
if errorlevel 1 goto :diff

echo  done %cnt%/%N%

if %cnt% lss %N% goto :loop

echo.
echo All %N% rounds passed.
set FAILED=0
goto :cleanup

:diff
echo.
echo *** MISMATCH at round %cnt% ***
echo ----- input -----
type input.txt
echo ----- expected -----
type ans_out.txt
echo ----- got -----
type test_out.txt
set FAILED=1
goto :cleanup

:fail
```

```
echo.
echo *** FAILED at round %cnt% (compile or run) ***
set FAILED=1

:cleanup
del data.exe ans.exe test.exe >nul 2>nul
exit /b %FAILED%
```